

INTERNSHIP PROJECT REPORT

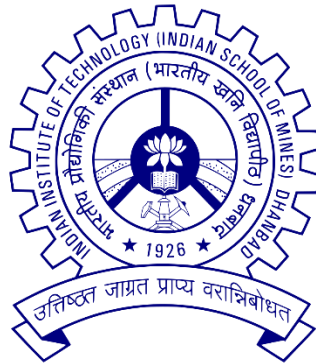
(WRIS-2025)

Path Planning for UAVs

Submitted in partial fulfilment of the requirement or the award of a certificate of
Winter Research Intern under WRIS-2025

Submitted to

**INDIAN INSTITUTE OF TECHNOLOGY (INDIAN SCHOOL OF
MINES) DHANBAD**



Submitted By

Nisha Priya

Btech- Computer Science and Engineering

(Specialization in Cyber Security and Cyber Defense) Sri Sri University

IIT (ISM) Dhanbad ID No.- 25WR0033

Under the supervision of Prof.

Chiranjeev Kumar

Professor and Head

Computer Science and Engineering

Indian Institute of Technology (Indian School of Mines), Dhanbad

CERTIFICATE

This is to certify that the work embodies in this dissertation entitled “Path Planning for UAVs” being submitted by “Nisha Priya” for partial fulfilment of the requirement for the award of “Internship Certificate” during the short term internship programme is a record of a Bonafide piece of work, which was carried out by her under my supervision and guidance in the Department of Computer Science and Engineering of Indian Institute of Technology (Indian School of Mines), Dhanbad

Signature and Date
Head of Department.

Signature and Date
Supervisor.

DECLARATION

I, Nisha Priya, hereby declare that the work presented in this report titled Path Planning for UAVs is my own work and has been carried out under the guidance of Dr. Chiranjeev Kumar and Ms. Shaily Priyanka Bodra. This report has not been submitted elsewhere for any academic award.

Signature
Nisha Priya
25WR0033
WRIS'2025 Intern

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my internship guide Dr. Chiranjeev Kumar and project supervisor Ms. Shaily Priyanka Bodra for their invaluable guidance, constant encouragement, and insightful discussions throughout the course of this Winter Internship at the Indian Institute of Technology (Indian School of Mines), Dhanbad. Their clarity of thought and constructive feedback greatly contributed to my understanding of the subject and played a crucial role in the successful completion of this work.

I am also thankful to the faculty members of the Department of Computer Science and Engineering at IIT (ISM) Dhanbad for providing a stimulating academic environment and for their support during the internship period. I would like to acknowledge the administrative and academic facilities provided by the institute, which made this internship a valuable learning experience. Finally, I would like to express my sincere appreciation to my home institution , Sri Sri University, for providing me with a strong academic foundation and continuous encouragement throughout my academic journey.

Signature
Nisha Priya
25WR0033
WRIS'2025 Intern

ABSTRACT

Path planning is a fundamental and crucial problem in robotics and autonomous navigation where a UAV must determine an efficient, feasible and collision free path from a starting position to the goal position in environments with obstacles. This project presents a detailed and comparative study of three path planning algorithms: **A-star (A*) , Rapidly exploring Random Tree (RRT) and Artificial Potential Field (APF).**

The effectiveness of path planning algorithms highly depends on the environment structure, obstacle density and computational constraints. Hence to showcase the performance of the algorithms, the evaluation is carried out in grid-based environments with varying complexities which are constructed using three different map sizes – **20x20, 60x60 and 100x100.**

For each map size, two obstacle configurations with different densities are considered: sparse and dense, which results in a total of six experimental scenarios for each algorithm.

In total 18 experiments were performed. To ensure fairness in the comparisons, a consistent number of iterations is maintained across all the algorithms in every scenario.

Performance is assessed using different quantitative and qualitative metrics which include completion time, success in path generation and obstacle collision occurrence.

The behavior of each algorithm during the path generation is visually analyzed for the better understanding of their operational characteristics.

The results show that A* performs efficiently in smaller and structured environments but has increased computational cost as the size of the map grows. RRT produces less optimal paths because of its sampling nature but demonstrates strong adaptability in complex and high dimensional spaces. APF offers fast and smooth path generation but as the result shows, it gets stuck in the local minima, particularly in dense obstacle configurations.

The evaluations and findings of this study using the experiments provide insights into the advantages/strengths and limitations of different path-planning approaches under varying environmental conditions.

In addition to the initial study, this work extends the evaluation to Rapidly Exploring Random Tree Star (RRT*) and a hybrid RRT* with Artificial Potential Field (RRT*+APF) approach. RRT* enhances standard RRT through cost optimization and rewiring, achieving asymptotically optimal paths, while the RRT*+APF hybrid incorporates potential field guidance to improve convergence speed and path smoothness. Both algorithms are evaluated under identical grid-based environments and obstacle densities for fair comparison. The results show that RRT* produces shorter and more optimal paths than RRT, whereas the RRT*+APF hybrid further reduces path length and convergence time in sparse and moderately dense environments, though local minima issues persist in highly cluttered scenarios. Overall, the extended analysis

highlights the trade-offs between optimality, computational efficiency, and robustness across classical, sampling-based, and hybrid path-planning methods.

PRELIMINARIES

The UAV path planning problem is formulated in a discretized 2D Cartesian workspace in a grid-based configuration space. Each cell in the grid is binary valued, where 0 denotes free space and 1 denotes obstacle regions.

For **A* path planning**, the grid is treated as a weighted graph, where each free cell represents a node. The path cost consists of a uniform step cost with a Euclidean-distance heuristic to guide the search toward the goal.

The **RRT algorithm** randomly selects points from the free spaces where movement is allowed (where there are no obstacles). It then extends the tree by a fixed distance from the nearest point and simultaneously keeps checking the path to make sure the new node-branch does not have to pass or cross through any obstacles.

In the **Artificial Potential Field (APF) approach**, the UAV drone moves under the influence of virtual forces generated by the goals and obstacles. All simulations are done in same environments, same starting and goal positions and same obstacle positions. It ensures that the outputs of the three methods can be compared fairly.

RRT* enhances standard RRT by introducing cost-based parent selection and rewiring, leading to asymptotically optimal paths with increasing iterations. **The RRT*+APF** hybrid incorporates artificial potential field guidance to accelerate convergence and improve path smoothness while preserving global exploration, though susceptibility to local minima remains in highly cluttered environments.

TABLE OF CONTENT

1.INTRODUCTION

2.BACKGROUND/RELATED WORK

- 2.1 A-Star (A*) Algorithm: Definition and Working**
- 2.2 Artificial Potential Field (APF) Algorithm: Definition and Working**
- 2.3 Rapidly-Exploring Random Tree (RRT) Algorithm: Definition and Working**
- 2.4 RRT* Algorithm: Definition and Working**
- 2.5 RRT*+ APF Algorithm: Definition and Working**

3. PSEUDOCODES FOR ALGORITHMS

- 3.1 A-star Algorithm: for 2D UAVs Path Planning**
- 3.2APF Algorithm: for 2D UAVs Path Planning**
- 3.3 RRT Algorithm: for 2D UAV Path Planning**
- 3.4 RRT* Algorithm: for 2D UAV Path Planning**
- 3.5 RRT* + APF Algorithm: for 2D UAV Path Planning**

4. EXPERIMENTS AND RESULTS

- 4.1 Environment setup**
- 4.2 Results and output analysis**

5. DISCUSSION

- 5.1 Performance Comparison**
- 5.2 Algorithm Behaviour**
- 5.3 Limitations and Trade-offs**
- 5.4 Observations**

6. CONCLUSION

7. FUTURE WORK

8. REFERENCES

9. APPENDICE

1.INTRODUCTION

UAVs use path planning methods for various purposes such as to figure out the location on its own, to understand the environment if they are free of obstacles or not, to choose the best path for reaching the goal, motion planning and control. It uses different techniques so that they do not need a human to control them. The whole process allows the UAVs to move efficiently and safely from starting position to the goal position.

1.1 Classification of UAV Path Planning Methods

UAV path planning techniques can be broadly classified into three major categories: Grid-based, Sampling-based and potential-field based.

1.1.a Grid-Based Path Planning

Grid-based methods divide the environment into grids or graphs. After which they use various algorithms like A*, Dijkstra, D*, Theta, Theta*, etc. to find the best path from the starting to the goal position

Real-World Example: A* algorithm is used in Robot Operating System Application: Help the UAVs to navigate on their own using ROS.

How it is used: The environment is represented as a grid map, where each cell is either free or has an obstacle. A* here helps in finding the best route/path from starting position to goal position which is efficient and also considers factors like distance travelled and closeness to the obstacles

1.1.b Sampling-Based Path Planning

Sampling-based methods randomly pick points in the free spaces where the UAVs can move. Then connect the points to create a path avoiding obstacles. Common algorithms under this method are- RRT, RRT*, Probabilistic Roadmap (PRM)

Real-World Example: Using RRT in NASA JPL UAV Research Application: Helps UAVs to operate on their own in unknown or complex environments.

How it is used: RRT selects points randomly in the UAV's space gradually growing a tree full of paths possible from starting to end position also avoiding the obstacles. This helps in finding collision-free paths which are essential in cluttered and complex environments

1.1.c Potential-Field-Based Path Planning

Potential field methods find path using the forces acting on the UAV's motion. Some common algorithms under this method are:- Artificial Potential Field(APF) and Vector Field Force(VFF).

Real-World Example: Using APF in DARPA projects with many UAVs Application: Help a swarm/group of UAVs in flying together safely avoiding obstacles and maintaining the formation.

How it is used: APF helps UAVs calculate forces in real time from its sensors. The attractive forces from the target pull the UAV and repulsive forces from the obstacles push it. This allows the swarm to navigate dynamically while avoiding obstacles and collisions and staying coordinated

2.BACKGROUND / RELATED WORK

Algorithms for path planning

In this work, A*, RRT, and Artificial Potential Field (APF) algorithms were implemented in Python and executed using the Spyder development environment. These implementations were used to simulate UAV path planning and visualize the generated paths under identical environmental conditions.

2.1 A-Star (A*) Algorithm: Definition and Working

The A*(A-Star) algorithm is one of the most widely used graph search algorithms for path planning in known environments.

It divides the environment into a grid assuming each cell center as a node. The algorithm finds the best path/route to the target by connecting node through which the vehicle can move avoiding the obstacles.

A* can be treated as Dijkstra's Algorithm but is smarter as it uses a heuristic (an estimate of the distance to the goal) to enhance the search which makes it faster yet finding the optimal path. The evaluation function in A* calculates the total cost of moving through nodes, helping to decide which path is to be taken next and is defined as:

$$f(n) = g(n) + h(n)$$

where:

- n is the current node,
- g(n) represents the actual cost from the start node to the current node,
- h(n) denotes the heuristic cost, which estimates the cost from the current node to the goal node.

The heuristic function h(n) is commonly computed using the Euclidean distance, expressed as:

$$h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$$

where:

- (x_n, y_n) are the coordinates of the current node,
- (x_g, y_g) are the coordinates of the target node.

By combining the actual path cost and the heuristic estimate, the A* algorithm efficiently searches for the shortest path which is collision- free from the start position to the target position.

2.2 Artificial Potential Field (APF) Algorithm: Definition and Working

The Artificial Potential Field (APF) algorithm is a classical path planning method/algorithm used to guide UAVs find the optimal path from the starting position to the target while avoiding obstacles in the environment.

The goal exerts an attractive force pulling the UAV towards it and the obstacles exert repulsive forces pushing the UAV away. The APF algorithm consists of two primary components:

- Attractive potential field which is generated by the goal position to pull the UAV towards the target. It is directly proportional to the increase of the distance from the goal and increases progress in the direction of the destination.
- Repulsive potential field which is generated by the obstacles to push the UAV away and prevents it from any type of collisions or damages. It is effective within a certain distance and provides obstacle avoidance

The resultant potential field of the APF algorithm is calculated as:

$$U_{APF}(x, y) = U_{att}(x, y) + U_{rep}(x, y)$$

where:

- $(U_{att}(x, y))$ is the attractive potential field,
- $(U_{rep}(x, y))$ is the repulsive potential field.

2.2.a Attractive Potential Field

The attractive potential field pulls the UAV toward the goal position and is defined as:

$$U_{att}(x, y) = \frac{1}{2}k_{att}D_g^2(x, y)$$

where:

- (k_{att}) is the attractive gain that determines the strength of the attraction by the goal/target,
- $(D_g(x, y))$ is the Euclidean distance between the UAV position and the goal/target position.

The Euclidean distance to the goal is given by:

$$D_g(x, y) = \sqrt{(x - x_{\text{goal}})^2 + (y - y_{\text{goal}})^2}$$

2.2.b Repulsive Potential Field

The repulsive potential field prevents the UAV from approaching obstacles and also preventing collisions and is defined as:

$$U_{\text{rep}}(x, y) = \begin{cases} \frac{1}{2}k_{\text{rep}} \left(\frac{1}{D_o(x, y)} - \frac{1}{r} \right)^2, & D_o(x, y) \leq r \\ 0, & D_o(x, y) > r \end{cases}$$

where:

- (k_{rep}) is the repulsive gain determining the strength of obstacle repulsion exerted by the obstacle.
- ($D_o(x, y)$) is the Euclidean distance between the UAV and the obstacle,
- (r) is the predefined safe distance between the UAV and the obstacle. The Euclidean distance to the obstacle is defined as:

$$D_o(x, y) = \sqrt{(x - x_{\text{obs}})^2 + (y - y_{\text{obs}})^2}$$

According to the above formulation, the repulsive potential field becomes active only when the UAV is within the safety radius(r) of an obstacle. When the UAV is at a safe distance ($D_o(x, y) > r$), no repulsive force is applied.

2.2.c Motion Control Using APF

The APF algorithm tries to make the total potential energy minimum/lowest at the goal/target position. Therefore, the UAV moves in the direction where the potential decreases the rapidly, which is determined by the negative gradient of the total potential field, which represents the resultant force acting on the UAV:

$$\mathbf{F}_{\text{APF}}(x, y) = -\nabla U_{\text{APF}}(x, y)$$

It represents the combined effect (resultant) of all forces acting on the UAV. This force directs it toward the goal and avoids the obstacles simultaneously.

2.3 Rapidly-Exploring Random Tree (RRT) Algorithm: Definition and Working

The Rapidly-Exploring Random Tree (RRT) algorithm is a sampling-based path planning method mostly used in complex environments where there are too many obstacles especially when the details of the environment are not known unlike grid-based methods.

RRT does not divide the environment into cells, unlike grid methods. Instead it selects points randomly in the free space where the UAV can move. It gradually builds a tree of possible paths by connecting the points step by step.

This allows RRT to explore the environment efficiently and find a feasible path even in uneven conditions.

The algorithm starts placing the tree at the starting position. Then it chooses points randomly within the free spaces (allowed area for the UAVs). For each randomly selected point, it finds the closest node that already exists in the tree. From this node, the tree is extended in the direction of the random point to a small fixed distance. New node is added only if there is no obstacle collision in the path between the two nodes.

By extending the tree little by little, RRT is able to rapidly explore the unexplored regions of the configuration space. This helps in finding a path even in the areas which are narrow and have complicated obstacle arrangements. The algorithm continues to expand until the goal is reached or an ending condition is applied.

RRT focuses on finding feasible paths instead of the best/shortest ones i.e. the first path it finds while exploring is not usually optimal. However the algorithm is said to be probabilistically complete i.e. as long as the algorithm runs and samples more points, the chance of finding an optimal path becomes very high. After the tree reaches the goal position, the final path is obtained by tracing the tree backward from the goal node to the start node.

2.4 Rapidly-Exploring Random Tree (RRT) - Star Algorithm: Definition and Working

Rapidly Exploring Random Tree Star (RRT*) is an optimization-based sampling algorithm designed for efficient path planning in continuous and complex environments. Unlike deterministic graph-search methods, RRT* incrementally constructs a tree by randomly sampling the free configuration space, enabling effective exploration in high-dimensional domains.

The algorithm begins at the start node and repeatedly samples random configurations from the obstacle-free space. For each sampled point, the nearest node in the existing tree is identified, and a new node is generated by extending toward the sample using a fixed step size. Collision detection is performed to ensure that the newly added edge does not intersect any obstacles.

RRT* improves upon the standard RRT by introducing cost-aware parent selection and a rewiring mechanism. Instead of connecting the new node to the nearest neighbor alone, RRT* searches for all nearby nodes within a defined radius and selects the parent that minimizes the cumulative path cost from the start. After insertion, neighboring nodes are rewired if the new

node provides a lower-cost path. This iterative optimization ensures that the solution converges toward the optimal path as the number of samples increases.

The path cost in RRT* is computed as:

$$c(n) = c(\text{parent}(n)) + \|n - \text{parent}(n)\|$$

where:

- n is the current node,
- $c(n)$ is the accumulated cost from the start node to n .

Through continuous sampling and rewiring, RRT* guarantees asymptotic optimality, making it suitable for large-scale and complex path-planning problems.

2.5 Rapidly-Exploring Random Tree (RRT) – Star + APF Algorithm: Definition and Working

The RRT*+APF algorithm is a hybrid path-planning approach that combines the global exploration and optimality of RRT* with the directional guidance of the Artificial Potential Field (APF) method. The primary objective of this integration is to improve convergence speed and path smoothness without compromising feasibility.

In this hybrid approach, artificial potential fields generate virtual forces that bias the RRT* tree expansion. An attractive potential pulls the sampling and extension process toward the goal, while repulsive potentials steer the tree away from obstacle regions. This guidance reduces random wandering and accelerates convergence, particularly in sparse and moderately cluttered environments.

The total potential function is defined as:

$$U(q) = U_{\text{att}}(q) + U_{\text{rep}}(q)$$

where q represents the UAV configuration. The attractive potential toward the goal is given by:

$$U_{\text{att}}(q) = \frac{1}{2} k_{\text{att}} \|q - q_g\|^2$$

and the repulsive potential due to obstacles is expressed as:

$$U_{\text{rep}}(q) = \begin{cases} \frac{1}{2} k_{\text{rep}} \left(\frac{1}{d(q)} - \frac{1}{d_0} \right)^2, & d(q) \leq d_0 \\ 0, & d(q) > d_0 \end{cases}$$

where q_g denotes the goal position and $d(q)$ is the distance to the nearest obstacle.

By incorporating potential field guidance into the RRT* expansion and optimization process, the RRT*+APF algorithm achieves faster convergence and smoother trajectories compared to standard RRT*. However, the influence of repulsive forces may still lead to local minima effects in highly dense obstacle environments.

3. PSEUDOCODES FOR ALGORITHMS

3.1 A-star Algorithm : for 2D UAVs Path Planning

Input:

Start node (x_{start}, y_{start}) , goal node (x_{goal}, y_{goal}) , a 2D grid map where free cells and obstacles are marked (0 = free space, 1 = obstacle), allowed movement directions, and a heuristic function that guesses/estimates the distance from a node to the goal $h(n)$.

Initialization:

The algorithm begins by placing the starting node into the open list: the list of nodes that are to be explored (implemented as a priority queue) and initializing an empty closed list.

The cost of the start node is set as $g(start) = 0$ i.e. the cost to reach the start node is zero, and its evaluation function is computed as $f(start) = g(start) + h(start)$ i.e. the total cost is calculated using the heuristic.

Two auxiliary data structures are also initialized: a came from map to store parent relationships and a visited nodes list for analysis and visualization; to keep track of the visited nodes and parent-child relationships for the reconstruction of path.

Main Loop:

While the open list is not empty, the node with the minimum evaluation value f is selected as the current node.

If the selected node is the goal node, the search stops there and the path is reconstructed.

Otherwise, the current node is marked as visited and added to the closed list.

All the valid neighboring nodes are then checked.

Nodes that lie in obstacle regions or already belong to the closed list are ignored/skipped. For each remaining /valid neighbor, the algorithm calculates a temporary cost to reach it from the current node.

If at all the new cost is lower than the cost stored before, the neighbor's path information and cost are updated. The node is then added to or updated in the open list for further exploration'

Path Reconstruction:

After reaching the goal, the final correct path is obtained. The algorithm traces back from the goal node to the start node and the resulting sequence is reversed to produce a valid path from start to goal.

Output: The algorithm outputs the final collision and obstacle-free path along with all the information regarding the explored nodes during the search.

Remarks:

1. The heuristic function may be chosen as either Euclidean (straight line distance) or Manhattan distance (grid-based distance using only horizontal and vertical moves).
2. A priority queue is always used to select the node with best/lowest cost which results in efficient and fast running of the algorithm.
3. In order to understand how the algorithm explored the environment, it is necessary to keep track of the visited nodes which is also used in analysis, debugging or visualization of the search process.

3.2 APF Algorithm: for 2D UAVs Path Planning

Input: The algorithm takes the starting position as (x_{start}, y_{start}) and goal position as (x_{goal}, y_{goal}) . A binary grid map is also used to represent free space (0) and obstacles (1). Several APF parameters control the movement of the UAVs and those include attractive gain k_{att} , repulsive gain k_{rep} , obstacle influence distance d_0 , step size, goal tolerance, and maximum number of iterations. More parameters are included for the detection in case the UAV is stuck somewhere (local minima) and to use actions for the UAV to escape.

Initialization: The UAV position is initialized at the starting position, and the path is started from that point. A distance map to the nearest obstacles is created that represents the distance from every point to the nearest obstacle. The direction in which the distance changes i.e. gradients are also computed and these gradients are later used to compute repulsive forces- the push force generated by the obstacles. A stagnation counter is also setup to monitor local minimum condition (the situations in which the UAVs get stuck).

Force Computation: The attractive force generated by the goal pulls the UAV toward it and is defined as:

$$F_{att} = k_{att}(goal - pos)$$

The repulsive force generated by the nearby obstacles pushes the UAV away from themselves and is activated only when the UAV is within the obstacle influence distance d_0 :

$$F_{rep} = \begin{cases} 0, & d \geq d_0 \\ k_{rep} \left(\frac{1}{d} - \frac{1}{d_0} \right) \frac{grad}{d^2}, & d < d_0 \end{cases}$$

where d represents the distance to the nearest obstacle and $grad$ denotes the gradient pointing away from the obstacle.

Main Loop: The algorithm iteratively calculates the total force acting on the UAV by adding the attractive and repulsive components. If the magnitude of the resulting force approaches zero (becomes very small), it is considered that the UAV is stuck in the local minima therefore stopping the algorithm.

Otherwise, the UAV position is slightly directed in the direction of the resultant force using a fixed step size. The updated position is to be kept within the grid boundaries/map limits and is ensured that it does not collide with any present obstacle. Each new valid position is then appended to the path.

After each movement, the distance to the goal is evaluated. If the UAV reaches an acceptable distance from the goal, the algorithm terminates or ends successfully. A stagnation check is also performed by monitoring changes in the distance to the goal across iterations. If the situation persists for a duration of time and not improving, a small random movement is applied to help the UAV escape local minima from being stuck

Output: The final output of the algorithm is a smooth and continuous path which is represented as a sequence of waypoints (connected points), from the start position to the goal position leading the UAV.

Remarks: The APF method is efficient, fast and well suited for real-time navigation of UAVs. However, careful tuning of parameters is required to avoid local minima because poor tuning can cause the UAV to get stuck in those points, particularly in environments with closely spaced obstacles.

3.3 RRT Algorithm : for 2D UAV Path Planning

Input: The algorithm is given a start position defined as (x_{start}, y_{start}) and a goal position defined as (x_{goal}, y_{goal}) . A binary grid map is also used to represent the free (0) and obstacle (1) regions, a maximum number of iterations N , limits for how long the algorithm runs and a fixed step size p that controls tree expansion/how far the tree grows.

Initialization: The RRT is initialized by creating a root/first node of the tree at the starting position. This node forms the initial tree structure. The position of the goal node is defined but not inserted into the tree at this stage. An empty list is created to store the waypoints/path points, and the total path length is initialized to zero.

Tree Expansion: The algorithm samples a point randomly from the free configuration space for each iteration up to the maximum limit N .

Then the node closest to this sampled point in the tree is found using a distance metric. From the node found, the tree is extended in the direction of the sampled point by a fixed step size ρ , that generated a new candidate node.

The path between the new node and the nearest node is examined for collisions. If the path in any way intersects any obstacles, the node is discarded.

Otherwise, the new node is added to the tree as a child of the nearest node. After a successful extension, the distance between the new node and the goal position is computed. If the goal lies within a distance ρ , the goal node is connected to the tree and the expansion process is terminated.

Path Reconstruction: Once the goal is connected to the tree, the final path is obtained by backward tracing from the goal position to the start position through the tree. The start position is inserted at the beginning of the path to produce a complete sequence of waypoints (connected points). The total path length is estimated using the step size and number of segments. Output: The algorithm produces the output that is a sequence of waypoints representing a collision-free path from the start position to the goal, along with the total path distance.

Remarks: RRT efficiently explores large and complex spaces/environments through random sampling and incremental and gradual tree growth. Though it is feasible in finding the valid path quickly, the path usually is not the shortest and may also need additional smoothing to improve quality

3.4 RRT-Star Algorithm : for 2D UAV Path Planning

Input

Start position (x_{start}, y_{start}) , goal position (x_{goal}, y_{goal}) ,
binary grid map (0: free, 1: obstacle),
maximum number of iterations N ,
fixed step size ρ ,
rewiring radius r .

Initialization

Create the root node of the tree at the start position (x_{start}, y_{start}) .

Initialize the cost of the start node to zero.

Define the goal position but do not insert it into the tree at this stage.

Initialize an empty list to store the path waypoints and set the total path length to zero.

Tree Expansion

For each iteration up to the maximum limit N :

1. Randomly sample a point from the free configuration space.
2. Find the nearest node in the tree to the sampled point using a distance metric.
3. Extend the tree from the nearest node toward the sampled point by a fixed step size ρ to generate a new candidate node.
4. Check the path between the nearest node and the new node for collisions.
 - If the path intersects any obstacle, discard the new node and continue to the next iteration.
5. Identify all existing nodes within a radius r of the new node.
6. Select the parent node from this neighborhood that results in the minimum path cost to the new node.
7. Add the new node to the tree with the selected parent and update its cost.
8. Rewire the neighboring nodes if connecting through the new node results in a lower path cost and the connection is collision-free.
9. Compute the distance between the new node and the goal position.
 - If the goal lies within a distance ρ , connect the goal node to the tree and terminate the expansion process.

Path Reconstruction

Once the goal node is connected to the tree, reconstruct the final path by tracing backward from the goal node to the start node through the parent links.

Insert the start node at the beginning of the path to form a complete sequence of waypoints.

Compute the total path length based on the distances between consecutive waypoints.

Output

A sequence of waypoints representing a collision-free path from the start position to the goal position, along with the total path length.

Remarks

RRT* efficiently explores large and complex environments through random sampling and incremental tree growth while continuously improving path quality through rewiring. Unlike RRT, RRT* converges toward an optimal path as the number of iterations increases, though at the cost of higher computational complexity.

3.5 RRT-Star + APF Algorithm : for 2D UAV Path Planning

Input

Start position (x_{start}, y_{start}) , goal position (x_{goal}, y_{goal}) ,
binary grid map (0: free, 1: obstacle),
maximum number of iterations N ,
fixed step size ρ ,
rewiring radius r ,
APF parameters: attractive gain k_a , repulsive gain k_r , obstacle influence radius d_0 .

Initialization

Create the root node of the tree at the start position (x_{start}, y_{start}) .
Initialize the cost of the start node to zero.
Define the goal position but do not insert it into the tree at this stage.
Initialize an empty list to store the path waypoints and set the total path length to zero.

Tree Expansion

For each iteration up to the maximum limit N :

1. Randomly sample a point from the free configuration space.
2. Apply Artificial Potential Field (APF) bias to the sampled point by computing:
 - An attractive force toward the goal position.
 - Repulsive forces from nearby obstacles within the obstacle influence radius.
3. Modify the sampled point using the resultant APF force to obtain a biased sample.
4. Find the nearest node in the tree to the biased sampled point using a distance metric.
5. Extend the tree from the nearest node toward the biased sampled point by a fixed step size ρ to generate a new candidate node.
6. Check the path between the nearest node and the new node for collisions.
 - If the path intersects any obstacle, discard the new node and continue to the next iteration.
7. Identify all existing nodes within a radius r of the new node.
8. Select the parent node from this neighborhood that results in the minimum path cost to the new node.
9. Add the new node to the tree with the selected parent and update its cost.

10. Rewire the neighboring nodes if connecting through the new node results in a lower path cost and the connection is collision-free.

11. Compute the distance between the new node and the goal position.

- If the goal lies within a distance ρ and the path is collision-free, connect the goal node to the tree and terminate the expansion process.

Path Reconstruction

Once the goal node is connected to the tree, reconstruct the final path by tracing backward from the goal node to the start node through the parent links.

Insert the start node at the beginning of the path to form a complete sequence of waypoints.

Compute the total path length based on the distances between consecutive waypoints.

Output

A sequence of waypoints representing a collision-free path from the start position to the goal position, along with the total path length.

Remarks

RRT* + APF combines the global exploration and optimality properties of RRT* with the directional guidance of Artificial Potential Fields. The APF bias accelerates convergence and improves path smoothness; however, the influence of repulsive forces may still introduce local minima effects in densely cluttered environments.

4. EXPERIMENTS AND RESULTS

This section presents the experimental setup and the results obtained by the process of path planning using different algorithms on various grid sizes.

4.1 Environment setup

The implementation, simulation and evaluation of the path planning algorithms- A*, APF and RRT were carried out in a controlled software environment as described below.

4.1.a Hardware Configuration

- Programming Language
- Language: Python 3.x
- Interpreter: CPython

4.1.b Development Environment

- IDE: Spyder (Anaconda Distribution)
- Execution Mode: Script-based execution (%runfile command)
- Environment Management: Conda virtual Environment (default Anaconda environment)

4.1.c Libraries and Frameworks

The following python libraries were used across all implementations:

Library	Purpose
numpy	Grid representation, numerical operations
matplotlib	Visualization of grids, obstacles, paths
heapq	Priority queue for A*
random	Random sampling in RRT
time	Execution time measurement

4.1.c Algorithm-Specific Setup

- A* Algorithm
 - i) Grid based environment – 20x20, 60x60, 100x100
 - ii) 4-connected movement model (up, down, left, right)
 - iii) Heuristic functions
 - a. Euclidean distance (20x20, 60x60)
 - b. Manhattan distance (100x100 optimized version)
 - iv) Visualization using incremental node expansion

Table: A* Algorithm Parameter table

Concept	Value
Grid resolution	20x20/60x60/100x100
Neighbor set	4 connected
Edge cost	1
Heuristic	Euclidean distance
Termination	Goal reached/ Empty set

Path length Unit	Grid steps
------------------	------------

It shows that A* is resolution-dependent but parameter- independent

- APF-Method
 - i) Attractive potential toward the goal
 - ii) Repulsive potential from obstacles
 - iii) Continuous potential over grids
 - iv) Gradient based movement
 - v) Susceptible to local minima in dense environments

Table: APF Parameter Configuration Across grid sizes

Category	Parameter	20x20	60x60	100x100
Environment	Grid size	20x20	60x60	100x100
Attractive field	Attractive gain	1.0	1.0	1.0
Repulsive field	Repulsive gain	120	100-150	150
	Influence radius	3.5 cells	5-6 cells	8 cells
Motion control	Step size	0.25	2.0	2.0-2.5
	Goal Tolerance	0.8	1.0	1.0
Termination	Maximum iterations	10000	30000	50000
Local Minima Handling	Local Minima detection	No	Yes	Yes
	Movement threshold	----	0.02	0.02
	Stuck counter limit	----	100	150
	Random escape noise	None	+/- 1.5	+/- 1.5
Safety	Hard obstacle constraint	Stop on collision	Enforced	Enforced

The table demonstrates that

- i) APF $\neq$ graph algorithm
- ii) APF $\neq$ resolution independent
- iii) But here local geometry dominates behavior

Hence, even though the map size increases, the APF step size could not be increased proportionally because movement safety is governed by nearby obstacles than the overall environment size.

- RRT- Algorithm
 - i) Random sampling in the configuration space
 - ii) Tree expansion using nearest node selection
 - iii) Collision examination to improve path planning

iv) Path extraction using parent links

Table: Parameters for RRT Algorithm

Parameter	20x20 grid	60x60 grid	100x100 grid
Grid size	20x20	60x60	100x100
Max-iterations	25000	25000	25000
Step-size	2	2	2
Sampling method	Uniform	Uniform	Uniform
Nearest node	Euclidean	Euclidean	Euclidean
Termination	Goal reached	Goal reached	Goal reached

- RRT* Algorithm

- i) Random sampling in the configuration space
- ii) Tree expansion using nearest node selection with cost evaluation
- iii) Optimal parent selection and rewiring to minimize path cost
- iv) Collision examination during node extension and rewiring
- v) Path extraction using parent links from goal to start

Table: Parameters for RRT-Star Algorithm

Parameter	20x20 grid	60x60 grid	100x100 grid
Grid size	20x20	60x60	100x100
Max-iterations	25000	25000	25000
Step-size	2	2	2
Sampling method	Uniform	Uniform	Uniform
Rewiring radius	Adaptive	Adaptive	Adaptive
Nearest node	Euclidean	Euclidean	Euclidean
Cost metric	Path cost	Path cost	Path cost
Termination	Goal reached	Goal reached	Goal reached

- RRT*+APF Algorithm

- i) Random sampling in the configuration space guided by artificial potential fields
- ii) Tree expansion using nearest node selection with cost and potential-based bias
- iii) Attractive force toward the goal and repulsive forces away from obstacles
- iv) Optimal parent selection and rewiring under combined path cost and APF influence
- v) Collision examination during node extension and rewiring
- vi) Path extraction using parent links from goal to start.

Table: Parameters for RRT-Star + APF Algorithm

Parameter	20x20 grid	60x60 grid	100x100 grid
Grid size	20x20	60x60	100x100
Max-iterations	25000	25000	25000
Step-size	2	2	2

Sampling method	APF-Guided	APF-Guided	APF-Guided
Attractive potential	Enabled	Enabled	Enabled
Repulsive potential	Enabled	Enabled	Enabled
Rewiring radius	Adaptive	Adaptive	Adaptive
Nearest node	Euclidean	Euclidean	Euclidean
Cost metric	Path cost + potential	Path cost + potential	Path cost + potential
Termination	Goal reached	Goal reached	Goal reached

4.1.d Visualization Settings

- Obstacle Representation: Black regions
- Free Space: White regions
- Start Node: Green Marker
- Goal Node: Red marker
- Explored Nodes: Blue markers
- Final Path: Red line
- Waypoints: Yellow circle

4.1.e Performance Evaluation Metrics

- Execution Time
- Number of Iterations/Node expansions
- Path Length
- Number of Waypoints
- Goal Reachability

4.1.e Execution Platform

- **Execution command**
 - A***
 - `%runfile 'A-star(20x20).py' --wdir`
 - `%runfile 'A-star(60x60).py' --wdir`
 - `%runfile 'A-star(100x100).py' --wdir`

Similar was used for APF, RRT, RRT* and RRT*+APF scripts

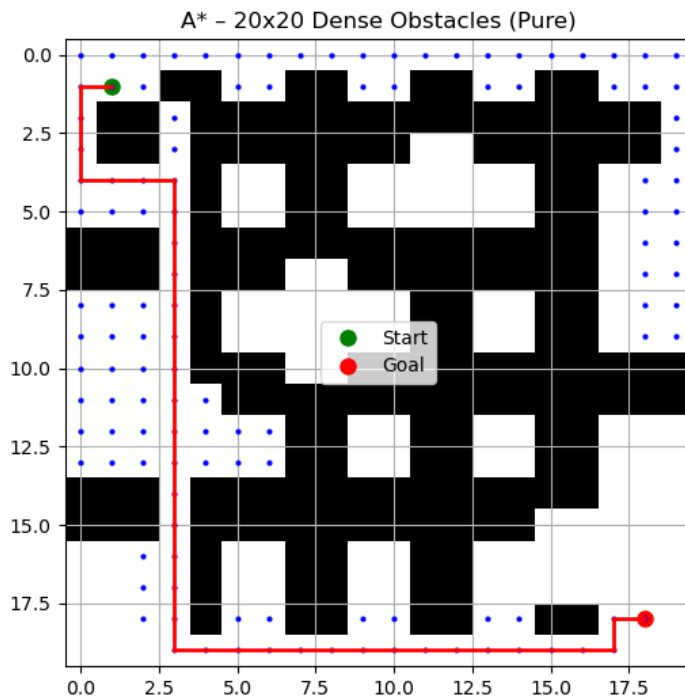
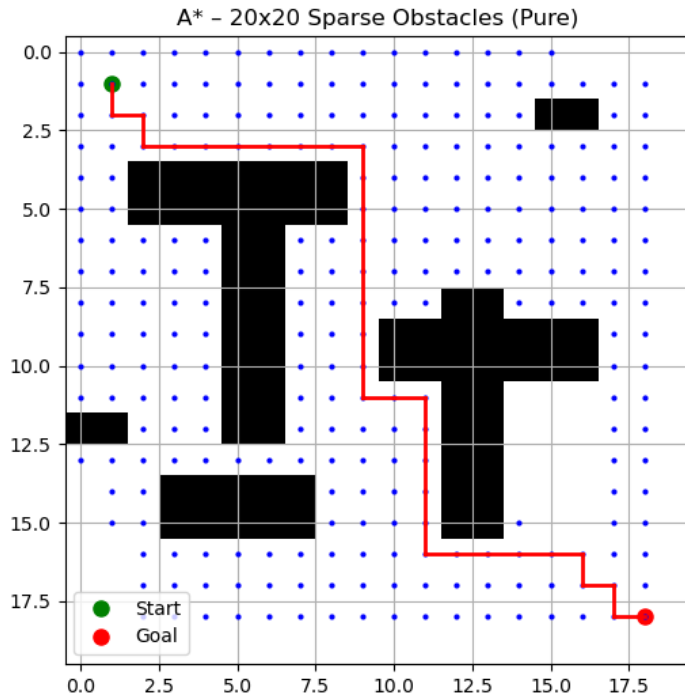
The above setup ensured consistent and fair evaluation of A*, APF and RRT algorithms across different grid sizes and obstacle densities. The environment was sufficient to handle large-scale simulations without the need for external libraries or hardware acceleration.

4.2 Results and output analysis

4.2.a A* Algorithm

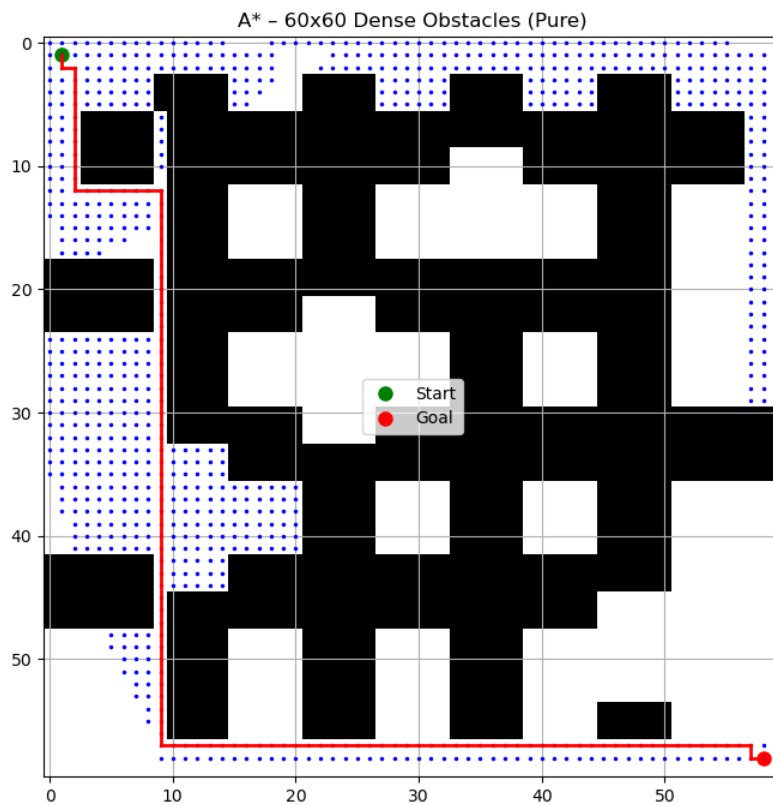
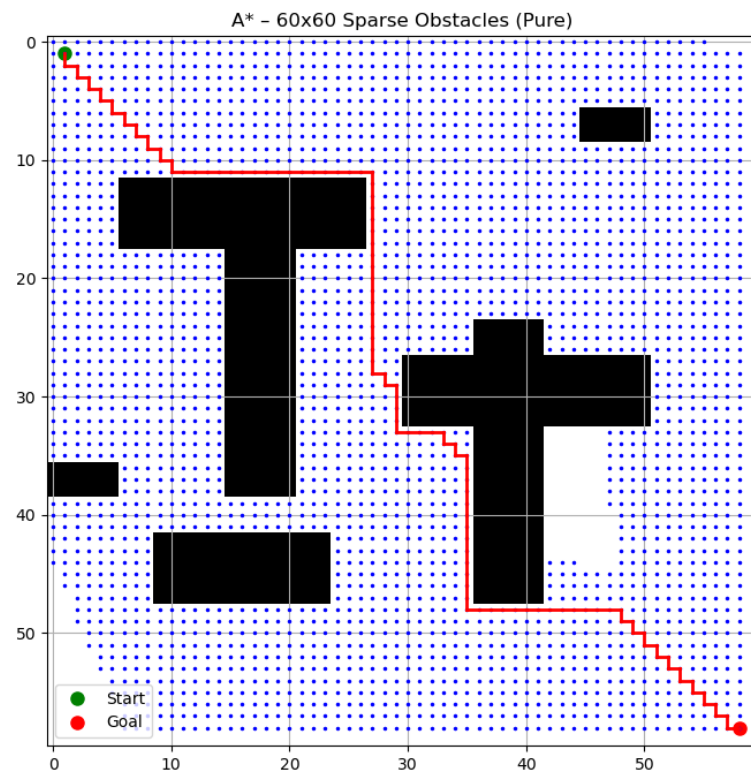
1) 20x20 grid

1.(a) Sparse Obstacles and Dense Obstacles



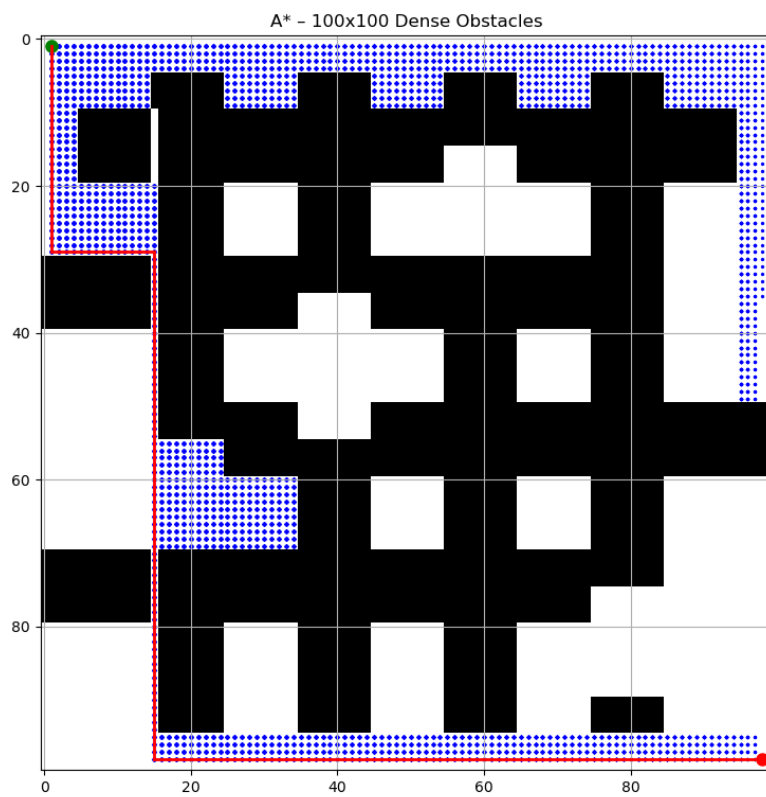
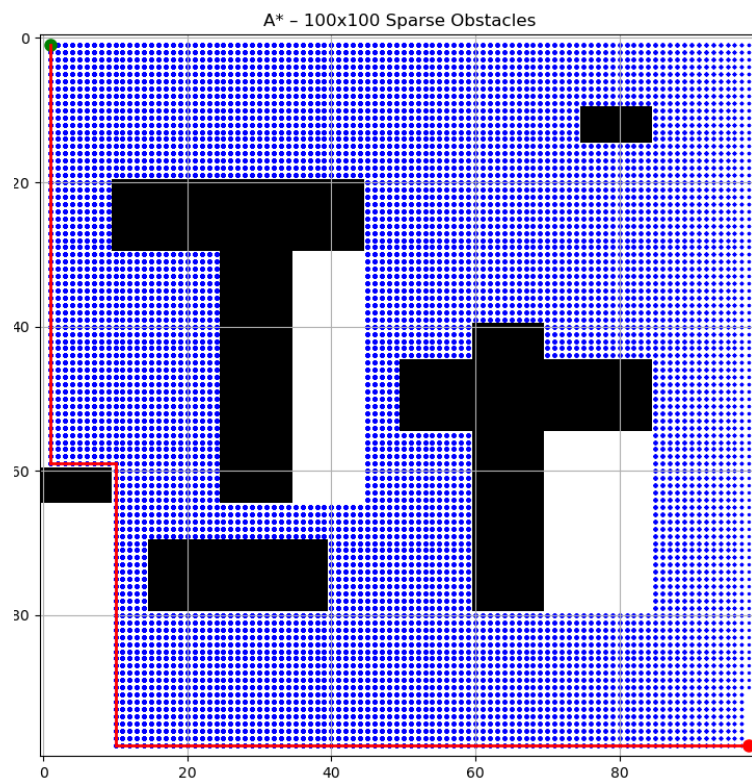
2) 60x60 grid

2)(a). Sparse Obstacles and Dense Obstacles



3)100x100 Grid

3(a).Sparse Obstacles and Dense Obstacles



Working of A* in the following grids:-

1. A* begins from the green start cell
2. It evaluates nearby cells and estimates how close they are to the red goal
3. Cells that appear more promising are explored first and marked as blue dots
4. When A* reaches obstacle cells (black), it avoids them and searches around them
5. The search spreads outward while still moving toward the goal
6. Some areas have more blue dots because alternative routes were evaluated
7. Once the goal cell is reached, the search stops
8. The final red path is formed by connecting the best sequence of explored cells

• Quantitative Results Summary

(a)A* -20x20

Obstacle Density Time (s) Waypoints Path Length Goal Reached

Sparse	19.969	35	34	Yes
Dense	40.673	39	38	Yes

(b)A*-60x60

Obstacle Density Time (s) Waypoints Path Length Goal Reached

Sparse	335.343	115	114	Yes
Dense	182.576	115	114	Yes

Observation: Lower execution time of the algorithm despite of the larger grid size due to restricted search space that resulted in less node expansions.

(c)A* - 100x100 grid

Obstacle Density Time (s) Nodes Expanded Path Length Goal Reached

Sparse	84.889	6878	194	Yes
Dense	7.583	1668	194	Yes

Observation: Dense environments significantly reduced node expansions, resulting in much faster execution compared to sparse environments.

There were time reduction techniques applied in 100x100 code

a)Visualization throttling

```
DRAW EVERY = 30  
if step % DRAW EVERY == 0:  
    plt.scatter(expanded_x, expanded_y, s=2)
```

This prevented Spyder freezing, and there was speed improvement.

(b)Manhattan Heuristic (Instead of Euclidean)

$$h(n) = |x1 - x2| + |y1 - y2|$$

This was more suitable for 4-connected grid and it reduced the heuristic evaluation cost.

The performance improvement in the 100×100 case is due to optimization of visualization and heuristic computation, not due to a change in the A* algorithm itself.

As there were several changes made in the 100x100 grid, Direct time comparison across grid size is not valid; however qualitative behavior and path optimality comparisons remain same.

Observations:-

1) Visual and Behavioral Comparison: Across all grid sizes, A* demonstrates goal-focused behavior driven by the heuristic function.

i) Observation:

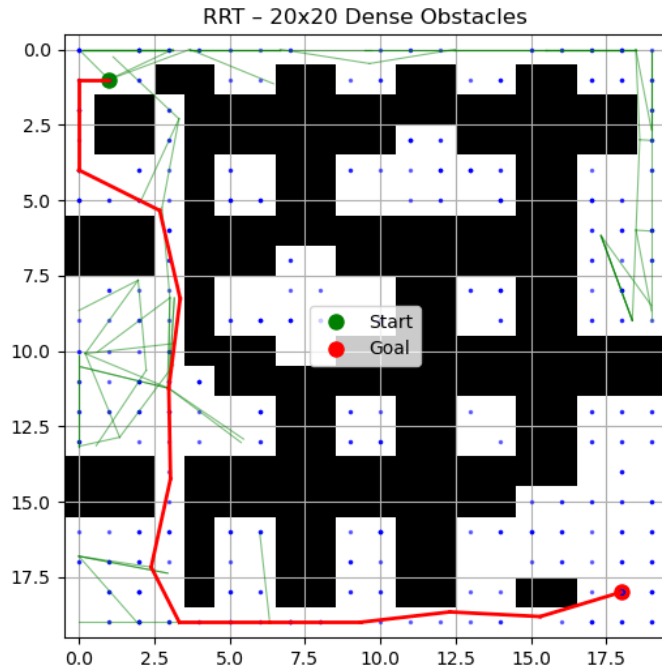
- (a) Blue nodes show systematic expansion toward the goal
- (b) Searches remain focused in sparse environments
- (c) In dense environments, A* expands laterally to discover corridors.

2) Sparse and Dense Obstacle reaction

- (i) Sparse Environment- Node expansion was less, there were few direct paths and there was minimal deviation from heuristic function.
- (ii) Node expansion was increased and there were multiple dead ends to examine.

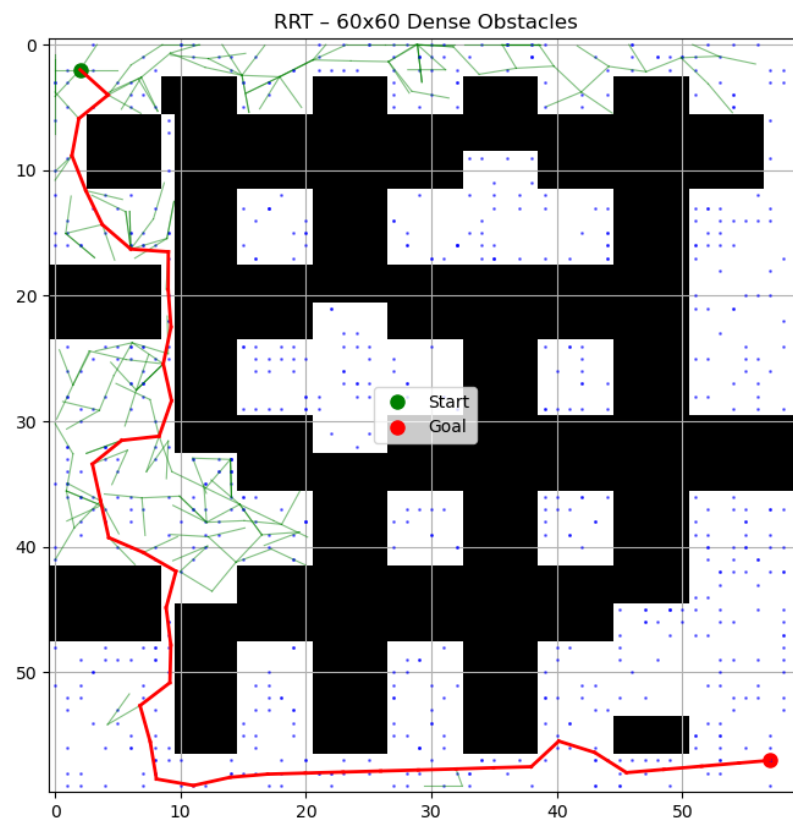
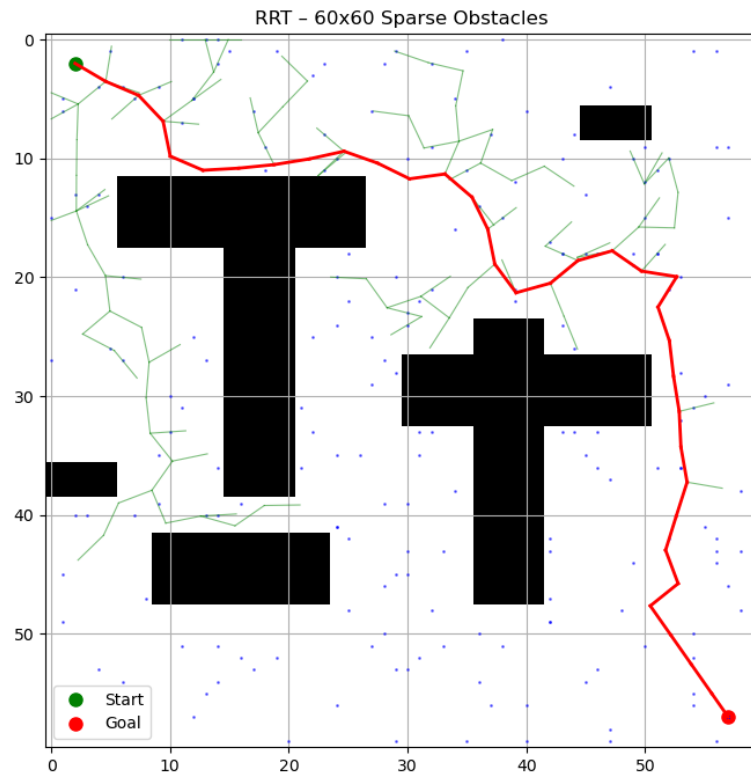
3) Execution Time

- (i) Dense environments take longer duration due to increased expansions (as observed from 20x20 grid and 60x60 grid).



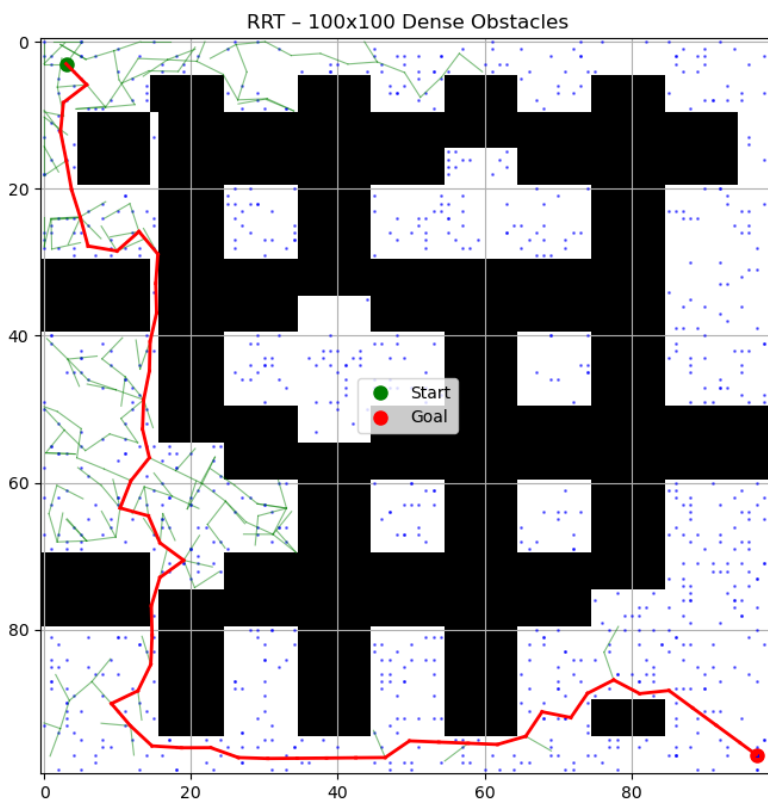
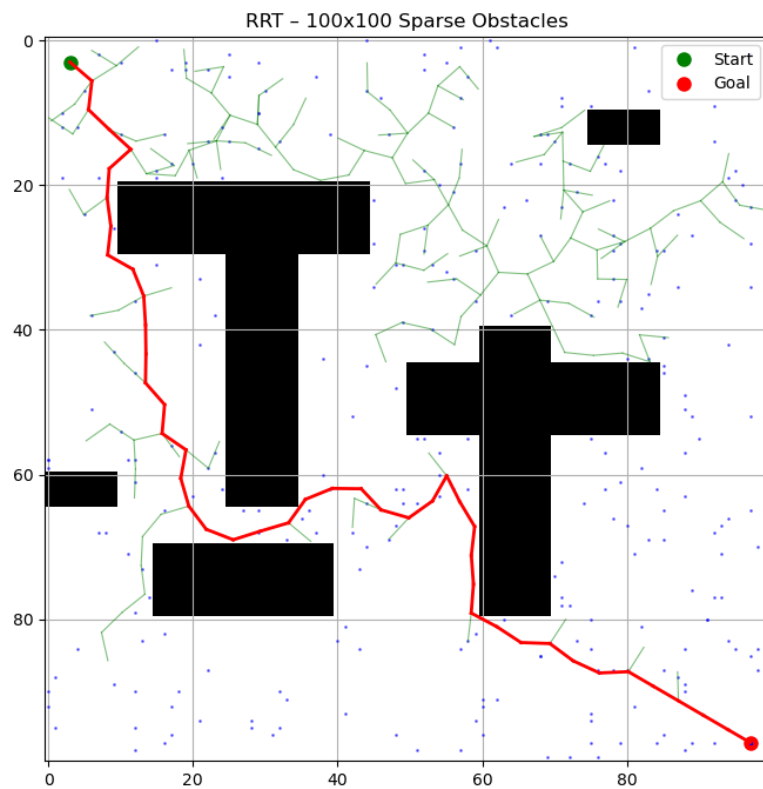
2) 60x60 grid

2)(a). Sparse Obstacles and Dense Obstacles



3)100x100 Grid

3)(a). Sparse Obstacles and Dense Obstacles



Working of RRT in the following grids:-

1. Each small dot represents a sampled robot position at a timestep. And these dots are created every time the robot moves a small step along the net force vector.
2. Blue dots appear more densely near obstacles, as the UAV's path changes direction more frequently when avoiding obstacles
3. In open space (no nearby obstacles), dots are spread further apart because movement is mostly straight.
4. The red line represents the final trajectory that reaches the goal while avoiding obstacles.

• Quantitative Results Summary

Direct comparison can be done in the RRT algorithm performance

Table: RRT Performance across Grid sizes and Obstacle densities

Grid Size	Obstacle Density	Iterations	Time(s)	Waypoints	Path length	Collision	Goal Reached
20x20	Sparse	40	2.243	14	39	No	Yes
20x20	Dense	526	62.808	14	39	No	Yes
60x60	Sparse	256	18.557	36	105	No	Yes
60x60	Dense	961	118.504	42	123	No	Yes
100x100	Sparse	354	27.924	46	180	No	Yes
100x100	Dense	1151	113.522	54	212	No	Yes

Observations:

1) Effect of Obstacle Density

- i) **Iterations :-** Increased number of rejections due to collision checks and dense environments require more iterations.
- ii) **Time of execution :-** Dense obstacle configuration require more time.

2) Path Comparison

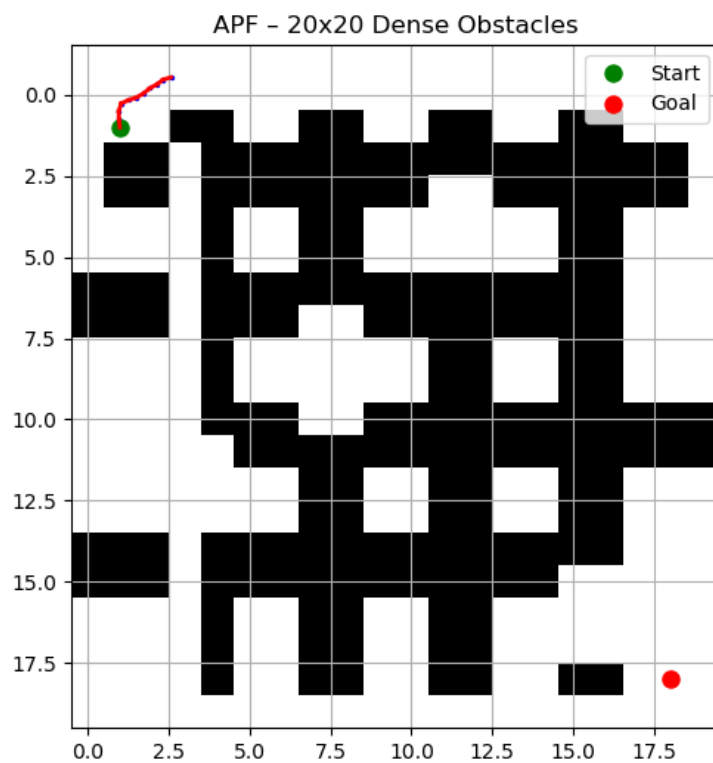
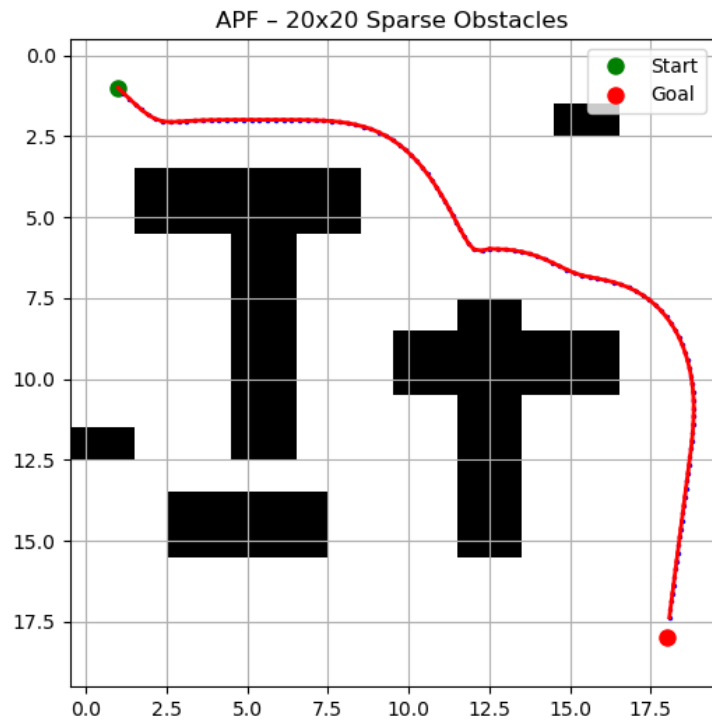
- i) Path length increases as the grid resolution increases
- ii) They do show zig-zag behavior
- iii) There are more number of waypoints in dense environments.
- iv) This shows that there were huge number of detours.

3) No obstacle collision was observed in any scenario.

4.2.c APF Algorithm

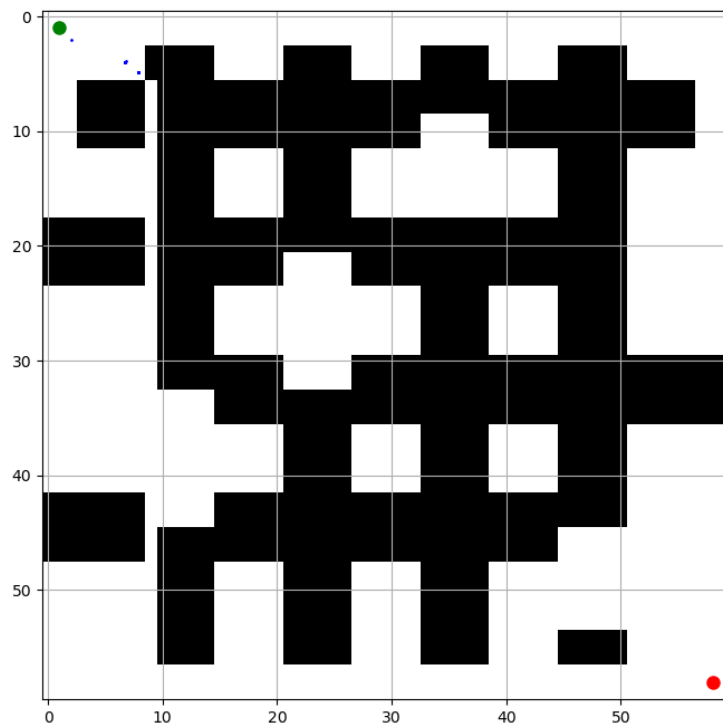
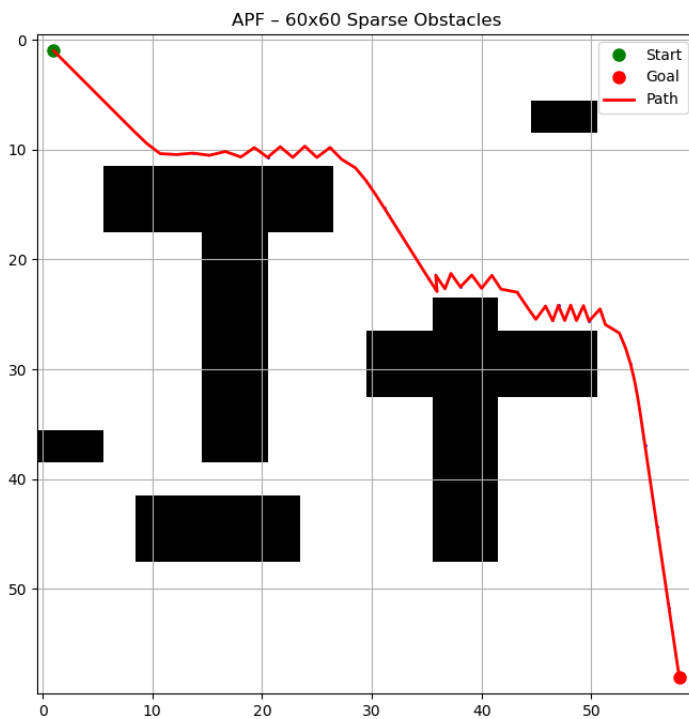
1) 20x20 grid

1.(a) Sparse Obstacles and Dense Obstacles



2) 60x60 grid

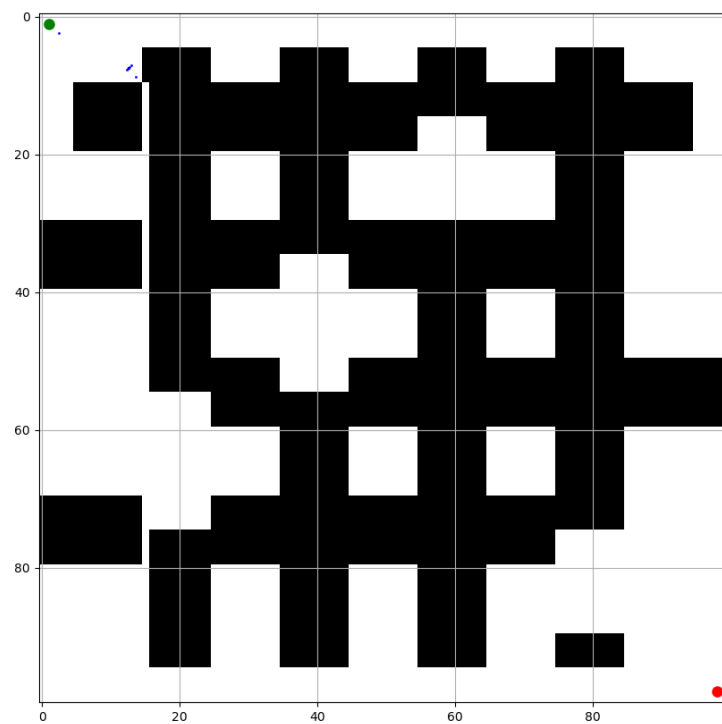
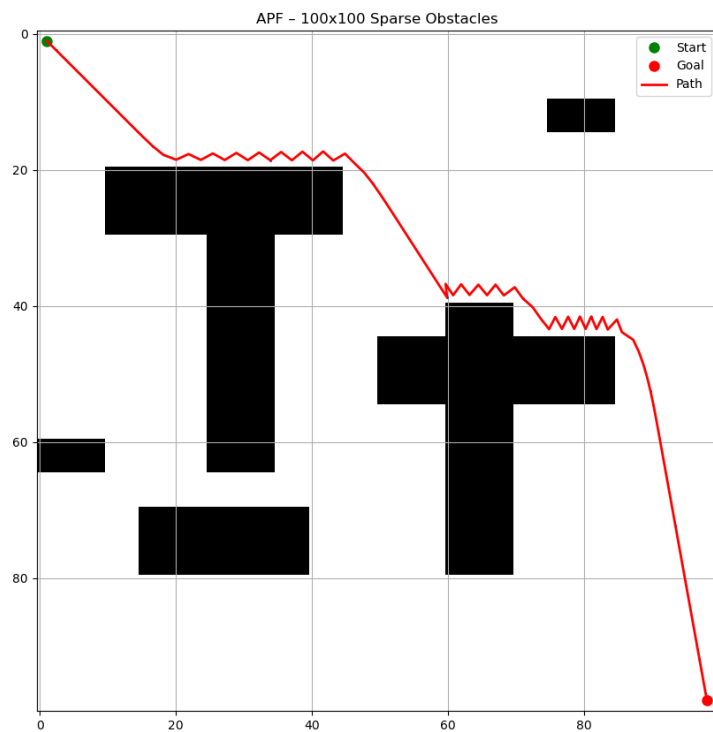
2)(a). Sparse Obstacles and Dense Obstacles



This is a NumPy–Matplotlib coordinate issue that the labelling wasn't shown and there is nothing wrong with the algorithm.

3)100x100 Grid

3(a). Sparse Obstacles and Dense Obstacles



This is a NumPy–Matplotlib coordinate issue that the labelling wasn't shown and there is nothing wrong with the algorithm.

Working of RRT in the following grids:-

1. The algorithm randomly generates points in the environments(the blue dots)
2. For each new random sample, the algorithm finds the nearest node in the current tree.
3. From the nearest node, it tries to move a small step toward the random sample.If the path doesn't intersect any obstacles: Add a new node at the position and draw a green line from the parent node to the new node.
4. When a node reaches withing a threshold distance of the goal, the goal node is added, and the path is traced back.

• Quantitative Result Summary

Grid Size	Density	Motion	Goal Reached	Reason
20x20	Sparse	Yes	Yes	Oscillation
20x20	Dense	Partial	No	Local minimum
60x60	Sparse	Yes	Yes	Oscillation
60x60	Dense	No	No	Local Minimum
100x100	Sparse	Yes	Yes	Oscillation
100x100	Dense	No	No	Local Minimum

The parameters were shown in 4.1.c Algorithmic specific parameter

Observations:-

1) Sparse and Dense Obstacles

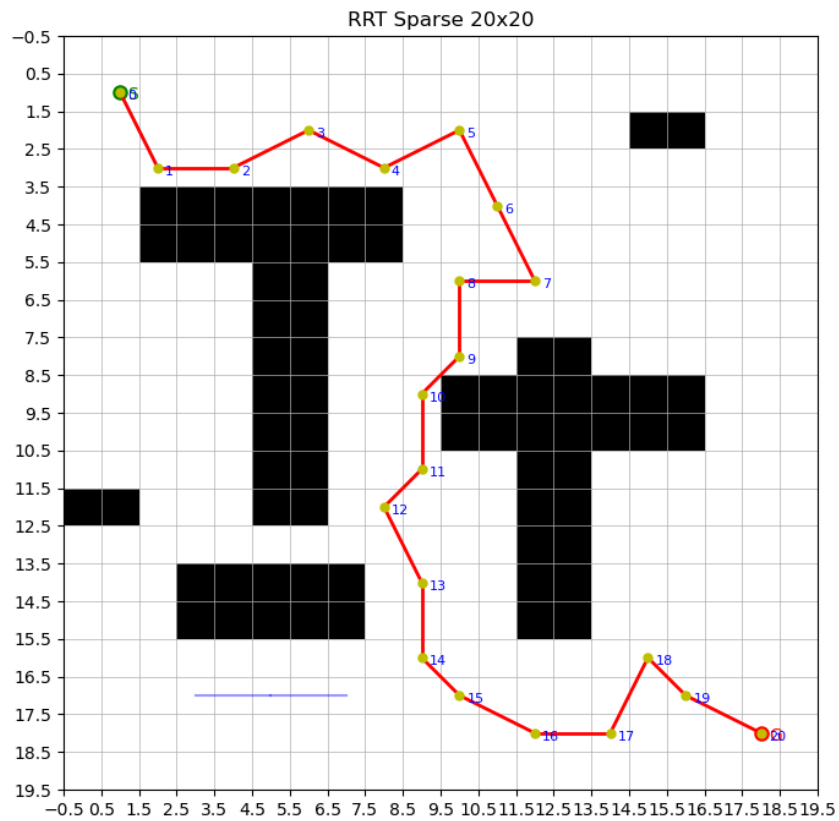
- a) In the sparse obstacle configuration, APF moves directly towards the goal and there are minor oscillations near the goal. Whereas in the dense obstacle configuration, the repulsive forces dominate and hence the UAVs are unable to escape the local minima.

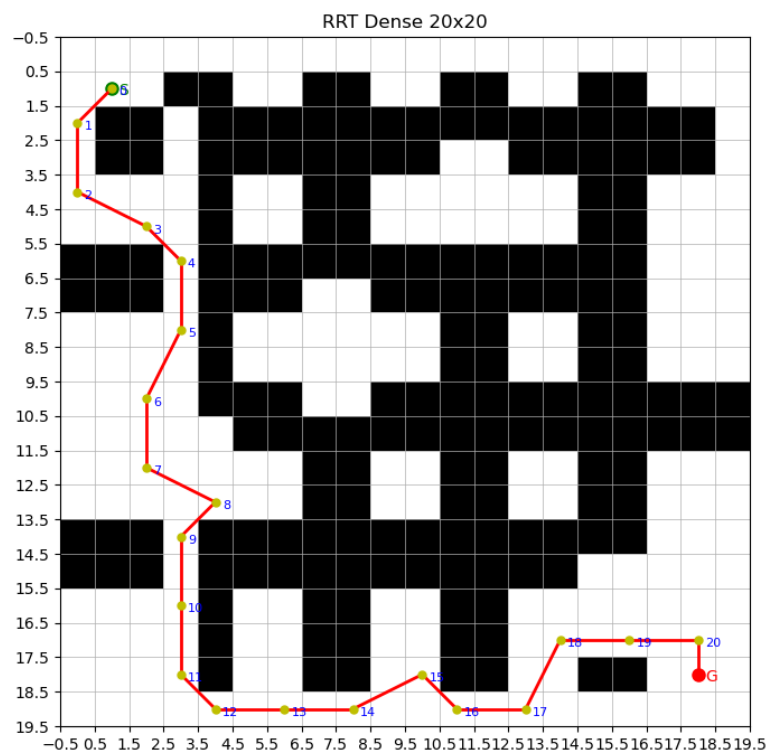
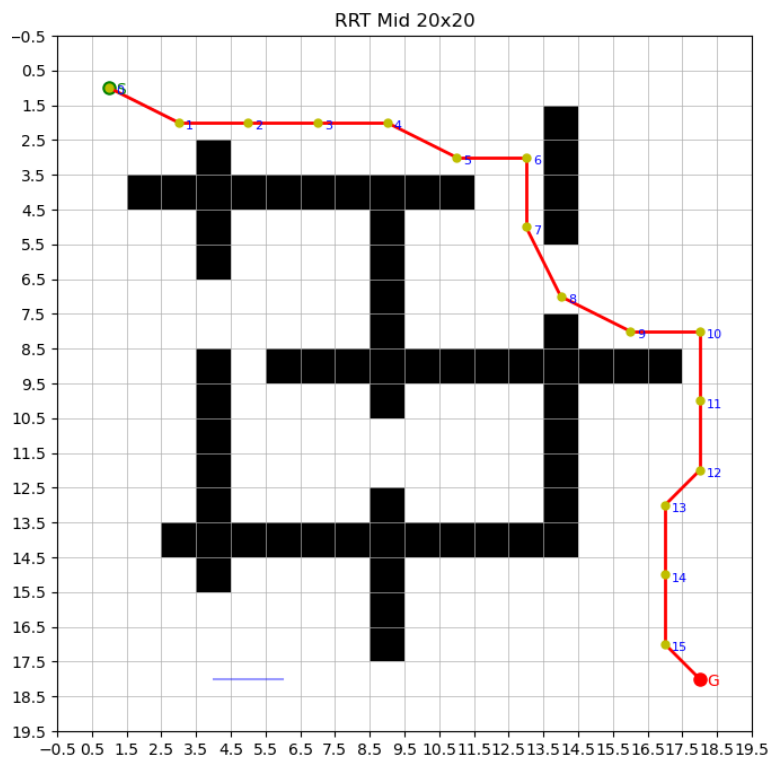
2) Oscillations:-

- b) These are caused by competing attractive and repulsive forces that almost cancel each other and these mostly occur near narrow passages.

2) Local Minima Problem:-

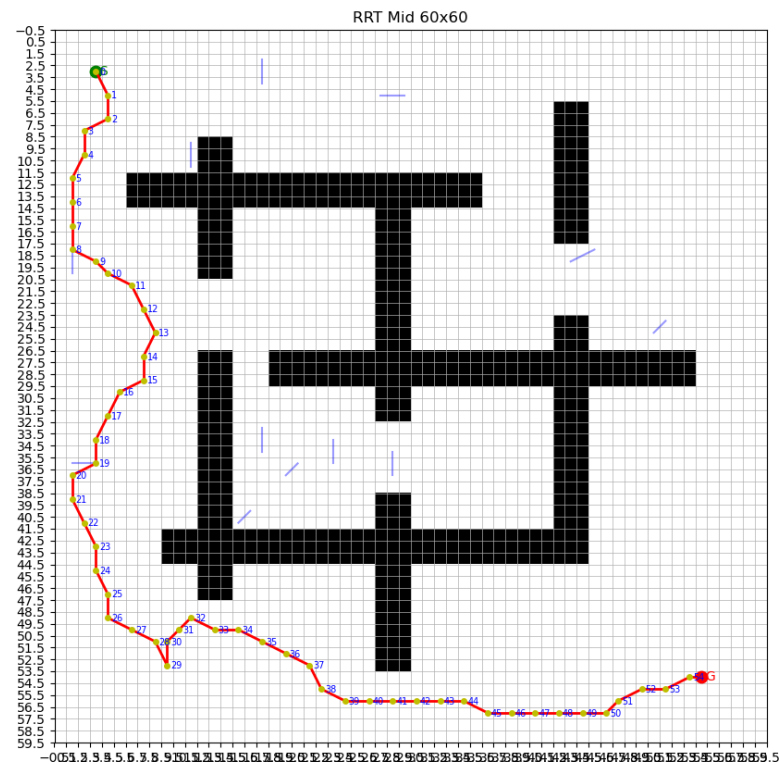
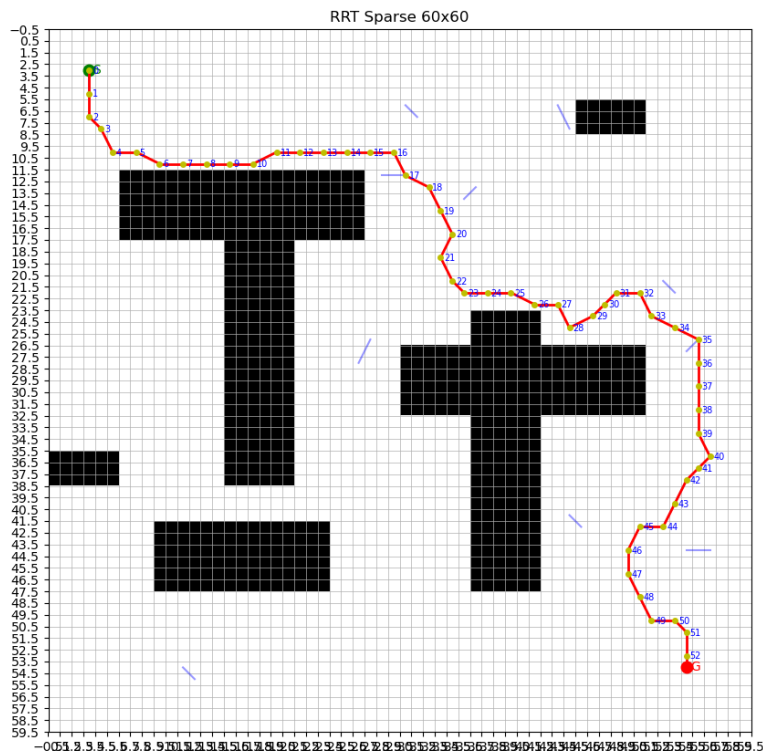
- c) Dense environments tend to trap the UAV in the local minima where the net force becomes zero but the goal is not reached.

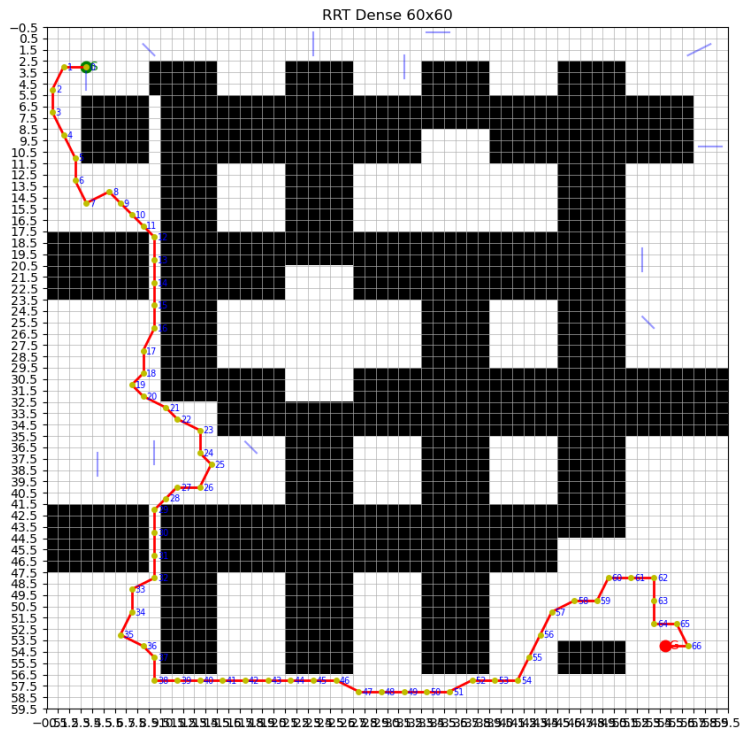




2) 60x60 grid

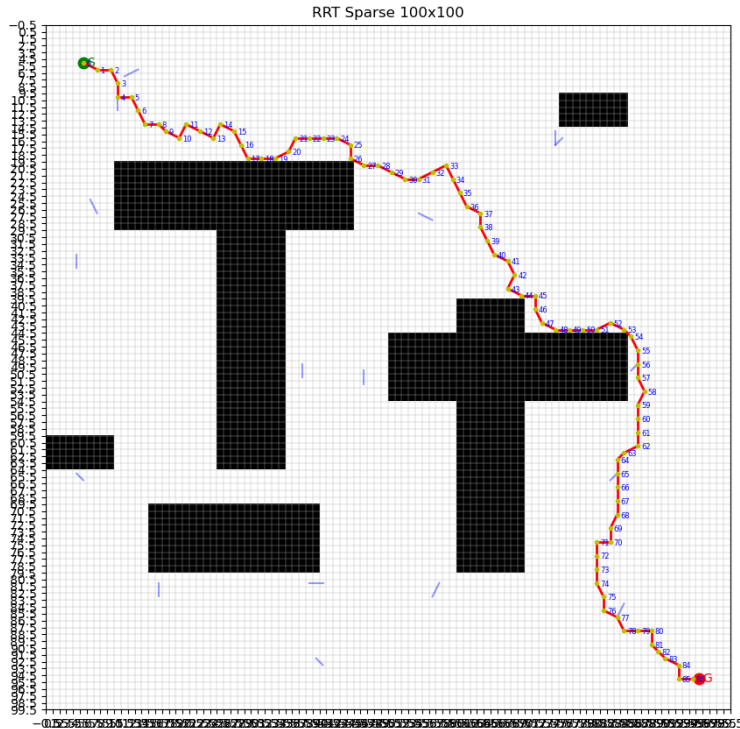
2.(a) Sparse Obstacles Mid Obstacles and Dense Obstacles



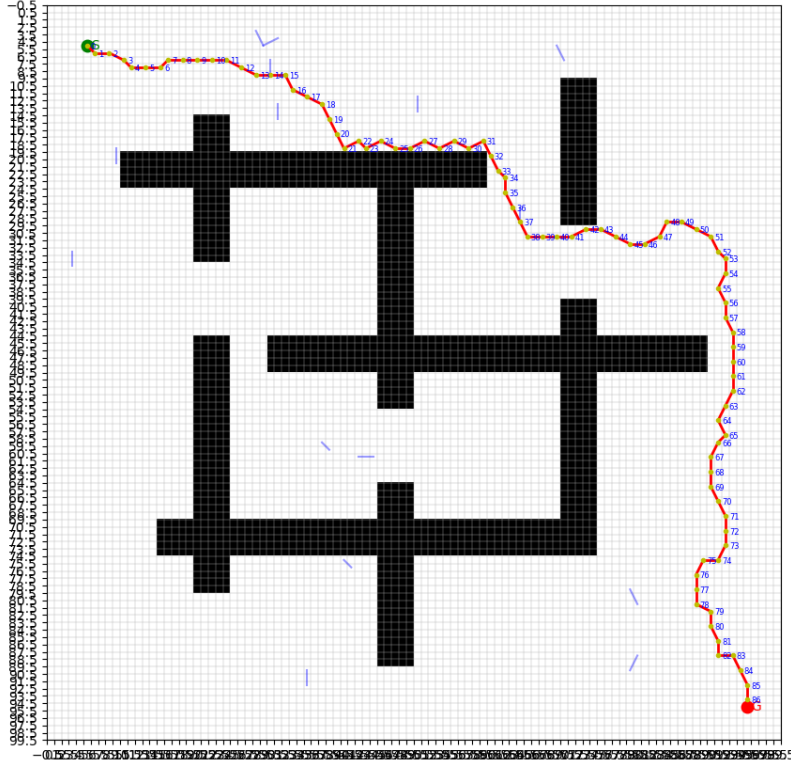


3) 100x100 grid

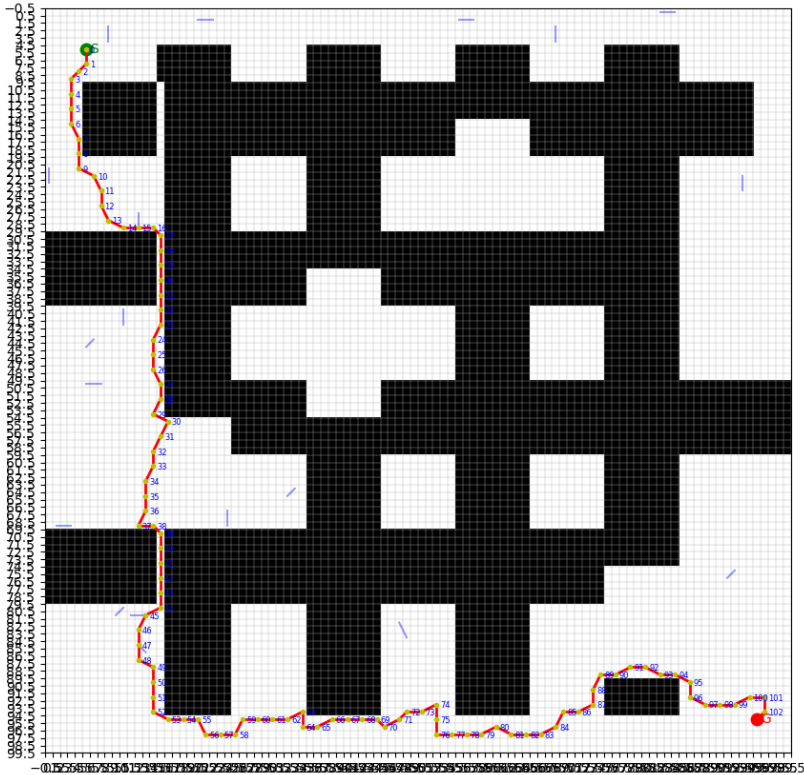
3.(a) Sparse Obstacles Mid Obstacles and Dense Obstacles



RRT Mid 100x100



RRT Dense 100x100



- Quantitative Result Summary

1.For Sparse Obstacle Density:

Run	20×20 Path	20×20 WP	60×60 Path	60×60 WP	100×100 Path	100×100 WP
1	40.02	22	105.86	54	180.22	88
2	36.90	20	108.85	53	200.11	96
3	34.78	19	100.15	50	192.11	94
4	32.36	16	112.22	57	207.83	102
5	36.60	20	108.33	53	198.59	94
6	37.55	20	104.09	52	193.12	92
7	31.42	16	103.44	51	215.89	103
8	32.07	17	103.09	52	184.35	91
9	33.12	17	99.57	49	198.47	96
10	33.90	19	108.15	53	186.82	92
11	32.36	17	99.68	49	185.41	90
12	44.25	22	94.04	48	178.64	87
13	34.60	19	111.62	55	187.95	92
14	35.37	19	113.45	56	182.22	88
15	34.36	17	114.04	56	196.01	95
16	32.90	18	107.50	53	185.82	90
17	32.83	17	108.15	54	166.06	81
18	36.72	19	120.51	59	182.24	90
19	38.54	20	108.44	55	195.70	95
20	32.36	16	102.38	49	195.23	94

Path Length (Standard Deviation)

- 20×20: 3.21
- 60×60: 6.13
- 100×100: 11.10

Grid	Best Path	Worst Path	Best WP	Worst WP
20×20	31.42	44.25	16	22
60×60	94.04	120.51	48	59
100×100	166.06	215.89	81	103

2.For mid Obstacle density:-

Run	20×20 Path	20×20 WP	60×60 Path	60×60 WP	100×100 Path	100×100 WP
1	31.77	17	113.03	56	179.52	88
2	33.48	18	103.09	52	198.77	98
3	38.13	20	105.45	54	196.42	96
4	35.42	18	118.85	59	195.70	95
5	40.54	22	113.45	56	170.58	84
6	34.65	18	105.97	52	212.65	103
7	37.48	20	111.75	56	212.05	102
8	35.55	19	104.02	50	184.64	90
9	42.13	22	110.38	54	213.65	105
10	33.89	18	119.04	61	196.40	94
11	32.42	18	109.68	55	201.23	98
12	34.18	18	99.14	49	189.83	93
13	35.95	19	110.27	54	189.77	94
14	35.19	19	113.16	58	174.34	85
15	37.95	20	121.21	59	198.65	96

Run	20×20 Path	20×20 WP	60×60 Path	60×60 WP	100×100 Path	100×100 WP
16	40.72	21	115.45	57	206.78	101
17	35.37	20	106.74	53	179.82	87
18	33.07	18	112.40	56	190.42	94
19	33.66	18	113.86	57	183.00	89
20	39.07	21	110.98	55	209.17	101

Path Length (Standard Deviation)

- 20×20: 2.93
- 60×60: 5.64
- 100×100: 12.91

Grid	Best Path	Worst Path	Best WP	Worst WP
20×20	31.77	42.13	17	22
60×60	99.14	121.21	49	61
100×100	170.58	213.65	84	105

3.For dense obstacle density:

Run	20×20 Path	20×20 WP	60×60 Path	60×60 WP	100×100 Path	100×100 WP
1	39.25	22	132.28	68	209.93	104
2	38.84	22	121.79	61	204.75	102
3	39.54	22	140.39	71	223.82	109
4	35.48	20	130.98	66	197.80	97
5	42.49	24	121.37	61	223.45	112
6	39.90	23	120.02	60	215.93	106
7	45.32	25	123.61	62	199.39	99

Run	20×20 Path	20×20 WP	60×60 Path	60×60 WP	100×100 Path	100×100 WP
8	38.84	22	117.09	59	204.35	103
9	40.71	22	115.43	58	218.45	105
10	40.84	23	129.97	66	207.57	102
11	39.55	22	130.04	67	204.46	102
12	44.19	25	118.20	59	222.64	109
13	41.07	22	141.86	71	206.69	102
14	43.02	25	121.09	63	209.63	105
15	40.13	22	129.91	64	215.81	107
16	37.79	22	122.62	62	222.04	108
17	38.84	22	127.61	64	214.76	106
18	37.37	21	130.27	65	203.86	101
19	38.13	21	123.92	63	220.58	109
20	36.96	22	132.98	68	218.11	108

Path Length (Standard Deviation)

- 20×20: 2.44
- 60×60: 7.28
- 100×100: 8.36

Grid	Best Path	Worst Path	Best WP	Worst WP
20×20	35.48	45.32	20	25
60×60	115.43	141.86	58	71
100×100	197.80	223.82	97	112

1.RRT Analysis

1.1 RRT – Sparse Maps

Observations (Sparse):

- RRT produces moderately long paths in sparse maps.
- Path length scales roughly linearly with grid size.
- Waypoints increase as the map grows, but fewer obstacles make paths less winding.
- Variance increases with grid size.

1.2 RRT – Mid Maps

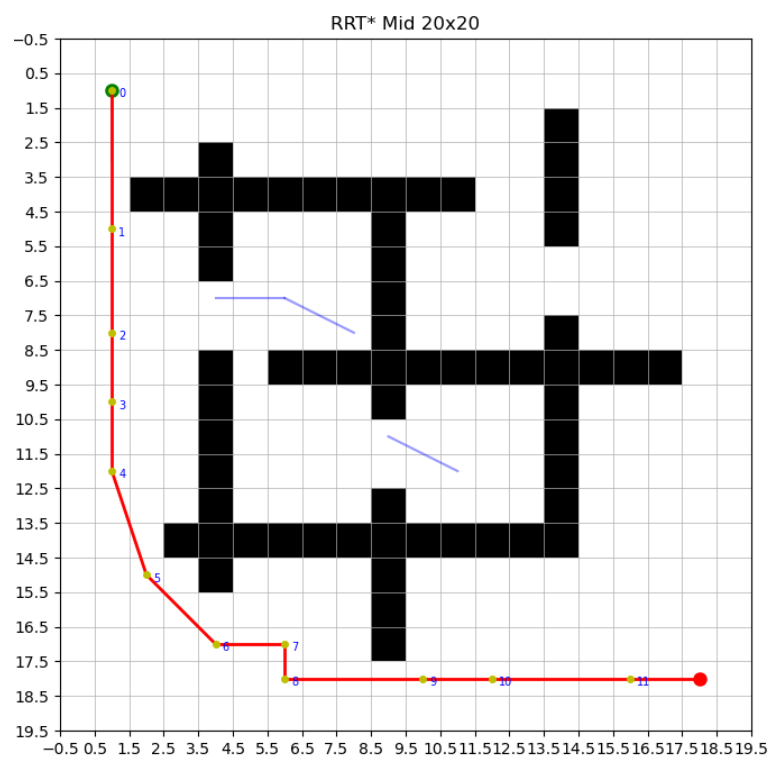
Observations (Mid):

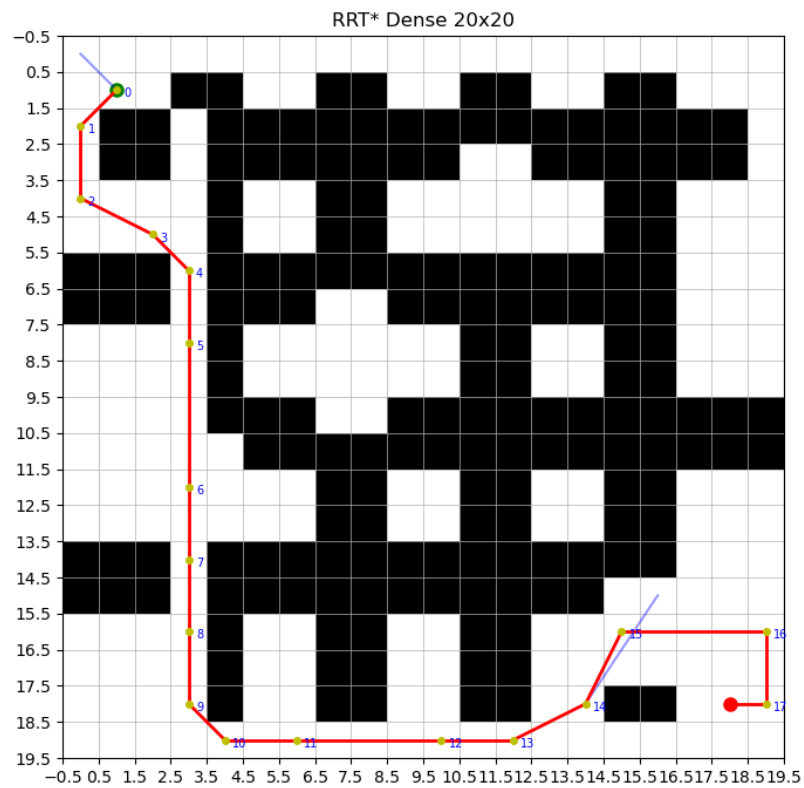
- Slightly longer paths than Sparse in 60×60 grids due to additional obstacles.
- 100×100 paths similar to Sparse—variability is slightly higher.
- Waypoints increase modestly as density grows, reflecting more winding paths.
-

1.3 RRT – Dense Maps

Observations (Dense):

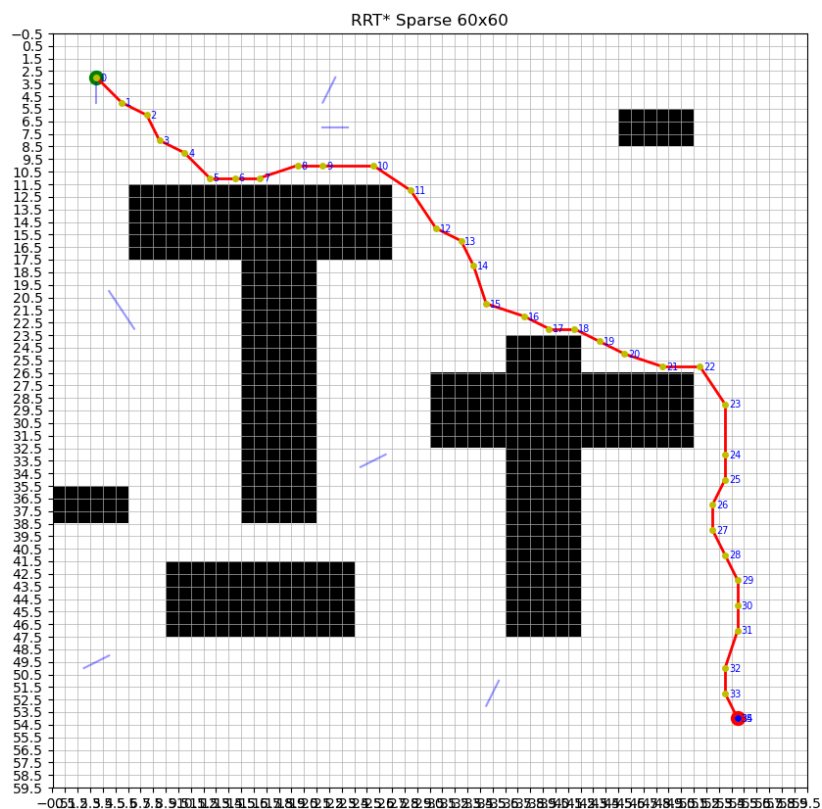
- Path length grows sharply due to obstacles; Dense maps are the most challenging.
- Waypoints increase significantly → paths are more winding.
- Variance is moderate for dense small grids but **less variable than mid for very large grids**, likely because almost all paths are forced to navigate the clutter similarly.

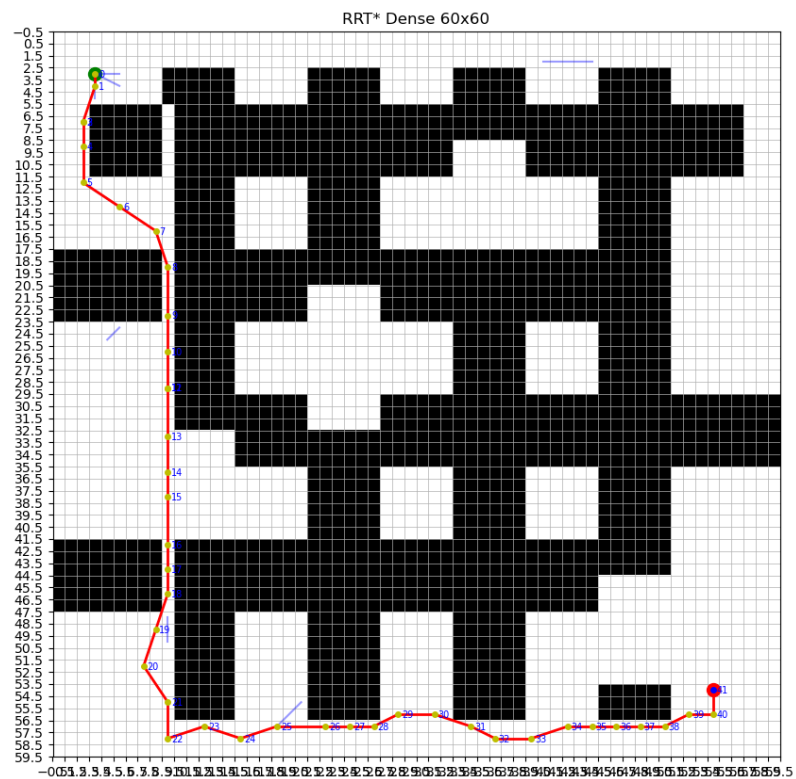
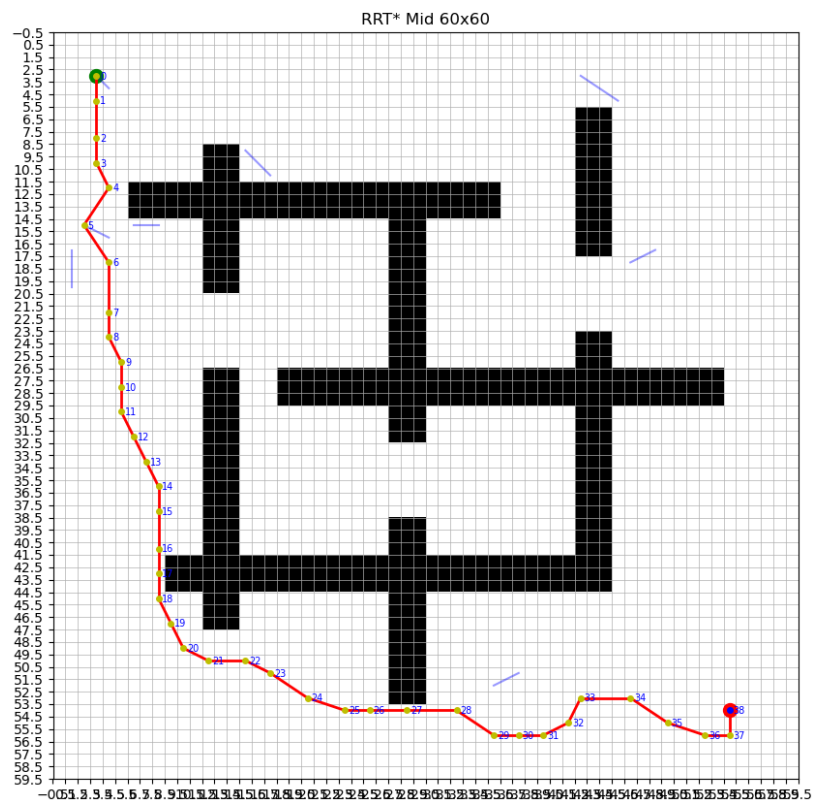




2) 60x60 grid

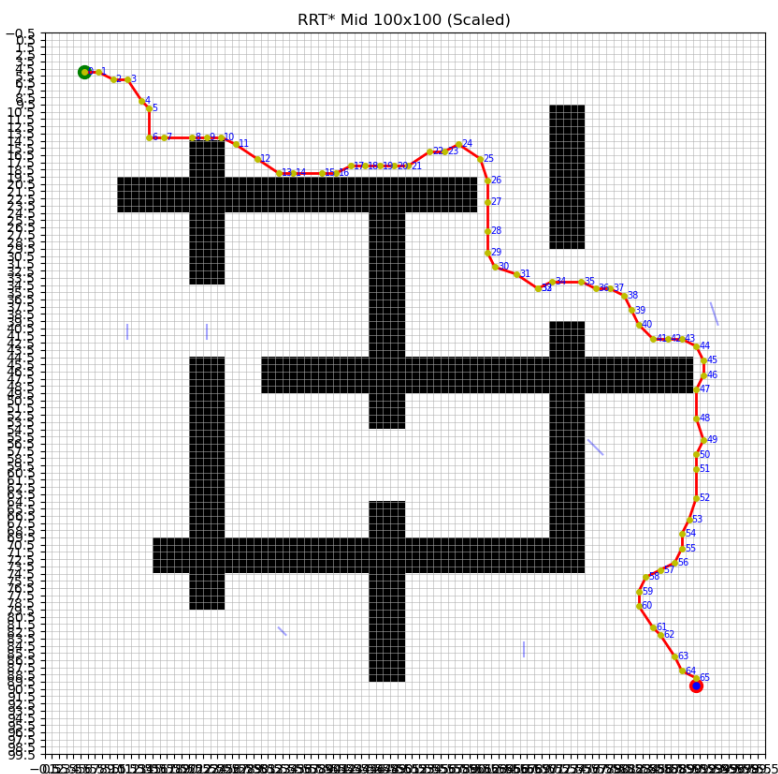
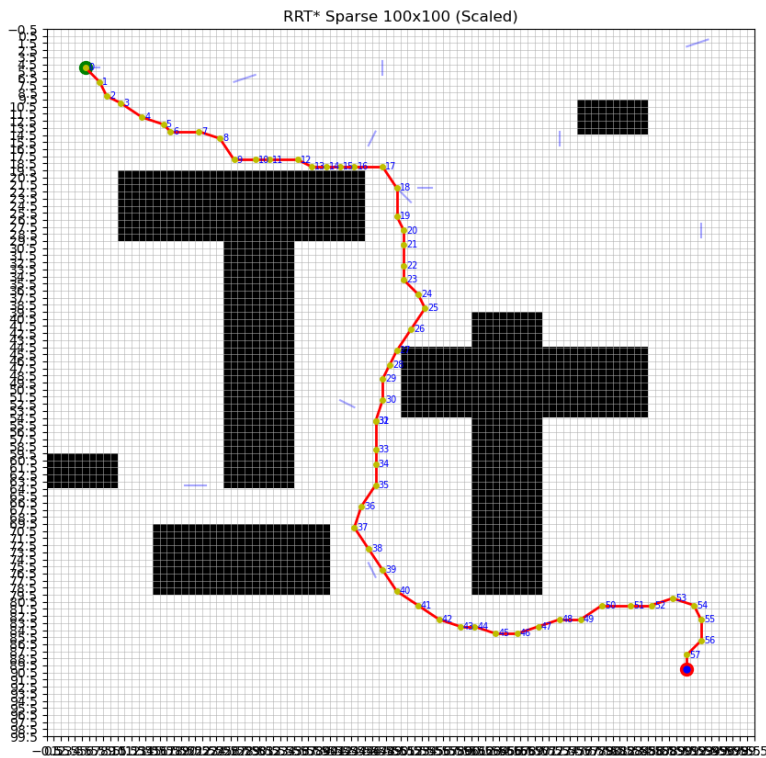
2.(a) Sparse Obstacles Mid Obstacles and Dense Obstacles

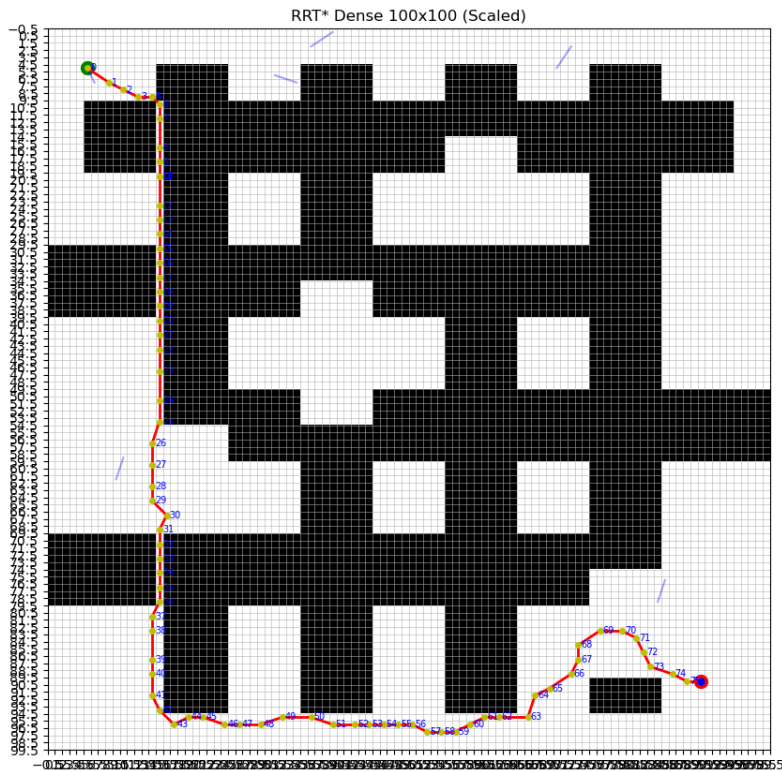




3) 100x100 grid

3.(a) Sparse Obstacles Mid Obstacles and Dense Obstacles





- Quantitative Results Summary

1.For Sparse obstacle density:

Run	20×20 Path	20×20 WP	60×60 Path	60×60 WP	100×100 Path	100×100 WP
1	30.68	13	88.12	36	159.46	62
2	30.20	13	85.25	35	177.39	73
3	28.69	13	100.58	41	164.14	63
4	29.24	12	93.20	33	168.68	59
5	30.92	14	99.28	38	143.84	56
6	29.01	12	90.57	37	159.87	62
7	29.68	14	90.89	38	161.86	66
8	28.71	15	90.35	37	150.81	57
9	27.59	13	90.80	31	143.91	60
10	31.20	12	90.24	37	161.92	65

Run	20×20 Path	20×20 WP	60×60 Path	60×60 WP	100×100 Path	100×100 WP
11	30.50	13	90.53	37	156.78	64
12	28.03	12	92.10	35	161.98	61
13	31.40	14	94.30	38	172.34	65
14	29.98	14	86.29	33	154.20	64
15	28.44	12	93.36	39	150.57	59
16	29.38	12	99.38	39	151.00	59
17	33.08	14	92.17	36	149.39	60
18	30.35	14	89.79	37	149.69	62
19	30.19	12	88.97	36	156.78	55
20	28.86	13	96.83	37	169.15	64

Path Length – Standard Deviation

- 20×20: 1.31
- 60×60: 4.18
- 100×100: 9.25

Grid	Best Path	Worst Path	Best WP	Worst WP
20×20	27.59	33.08	12	15
60×60	85.25	100.58	31	41
100×100	143.84	177.39	55	73

2.For Mid obstacle density

Run	20×20 Path	20×20 WP	60×60 Path	60×60 WP	100×100 Path	100×100 WP
1	31.99	13	98.95	39	155.56	62
2	32.23	14	91.97	36	165.61	59
3	32.32	14	95.46	36	153.47	58
4	31.20	12	101.80	42	167.11	67
5	31.23	16	98.15	39	170.13	64
6	33.55	15	94.95	35	172.61	66
7	29.63	13	92.91	36	156.60	61
8	32.21	14	106.28	44	159.88	59
9	32.03	14	94.81	38	168.61	60
10	31.58	13	97.81	38	152.40	60
11	31.58	12	107.43	43	156.73	61
12	31.20	14	93.50	36	171.07	65
13	31.27	14	103.05	38	172.36	66
14	31.61	15	96.93	35	153.00	58
15	31.27	13	97.05	41	165.40	59
16	32.19	14	93.29	38	158.39	62
17	31.55	15	104.13	43	156.73	60
18	30.68	15	101.24	39	170.28	66
19	31.30	15	95.86	36	159.54	61
20	32.05	14	99.48	36	157.39	60

Path Length – Standard Deviation

- 20×20: 0.78
- 60×60: 4.50

- 100×100: 7.06

Grid	Best Path	Worst Path	Best WP	Worst WP
20×20	29.63	33.55	12	16
60×60	91.97	107.43	35	44
100×100	152.40	172.61	58	67

3.For Dense obstacle density

Run	20×20 Path	20×20 WP	60×60 Path	60×60 WP	100×100 Path	100×100 WP
1	39.95	19	107.99	42	183.54	72
2	37.11	18	111.92	47	178.88	71
3	36.31	19	109.29	44	188.79	77
4	37.25	18	106.53	47	184.40	77
5	36.89	19	107.33	48	185.37	73
6	34.89	19	104.81	41	176.30	73
7	36.60	19	112.70	46	179.36	71
8	37.43	20	108.62	42	183.31	72
9	36.60	20	109.72	48	184.62	73
10	34.64	18	104.21	46	178.23	76
11	35.48	19	104.90	45	178.49	69
12	38.53	20	114.44	44	179.99	70
13	34.71	18	105.92	43	178.44	72
14	35.48	19	106.83	45	178.92	70
15	39.01	20	111.63	45	178.13	74
16	35.47	19	106.44	47	188.40	83
17	34.89	18	105.60	42	183.20	72

Run	20×20 Path	20×20 WP	60×60 Path	60×60 WP	100×100 Path	100×100 WP
18	36.46	17	107.37	46	183.83	73
19	36.82	18	105.64	47	179.11	72
20	36.78	19	105.85	47	180.70	68

Path Length – Standard Deviation

- 20×20: 1.44
- 60×60: 2.88
- 100×100: 3.58

Grid	Best Path	Worst Path	Best WP	Worst WP
20×20	34.64	39.95	17	20
60×60	104.21	114.44	41	48
100×100	176.30	188.79	68	83

1.RRT* Analysis

1.1 Sparse Maps

Observations:

- RRT* finds efficient paths even on sparse maps.
- Standard deviation grows with grid size (expected due to more exploration required).
- Path length increase is roughly proportional to grid size (~linear trend in sparse maps).

1.2 Mid Maps

Observations:

- Slightly longer paths than sparse maps, as expected due to **more obstacles**.
- Number of waypoints is slightly higher and paths are more winding.
- Standard deviation increases with density and grid size.

1.3 Dense Maps

Observations:

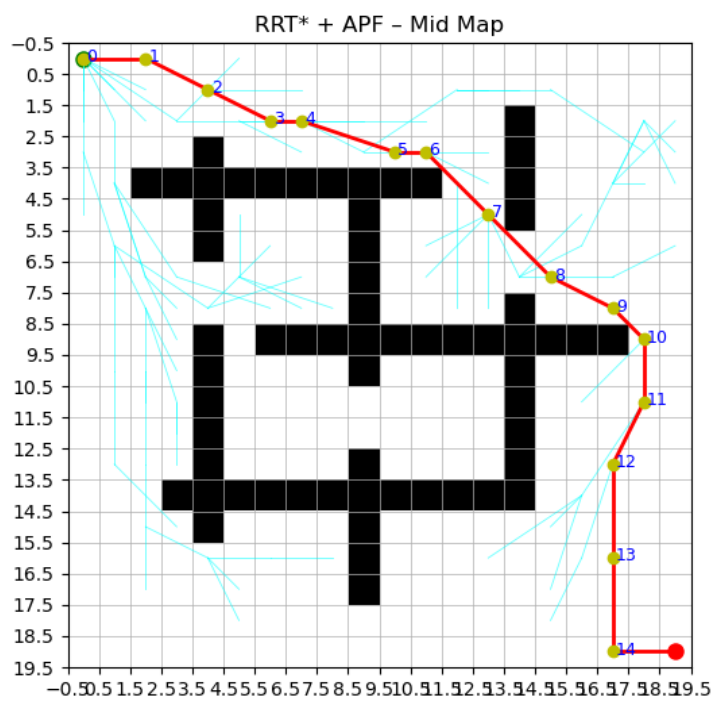
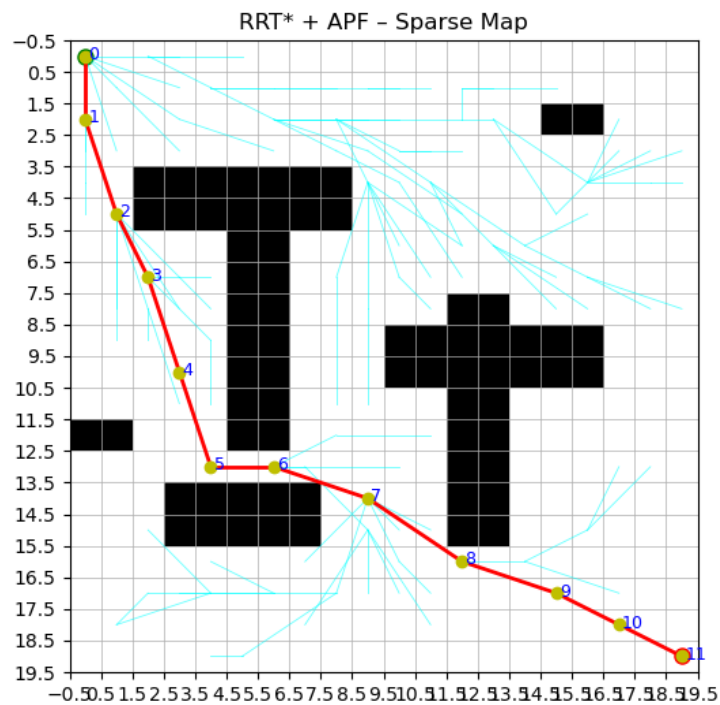
- Dense maps significantly increase path length and WP.
- RRT* still maintains **relatively low variance** despite density.

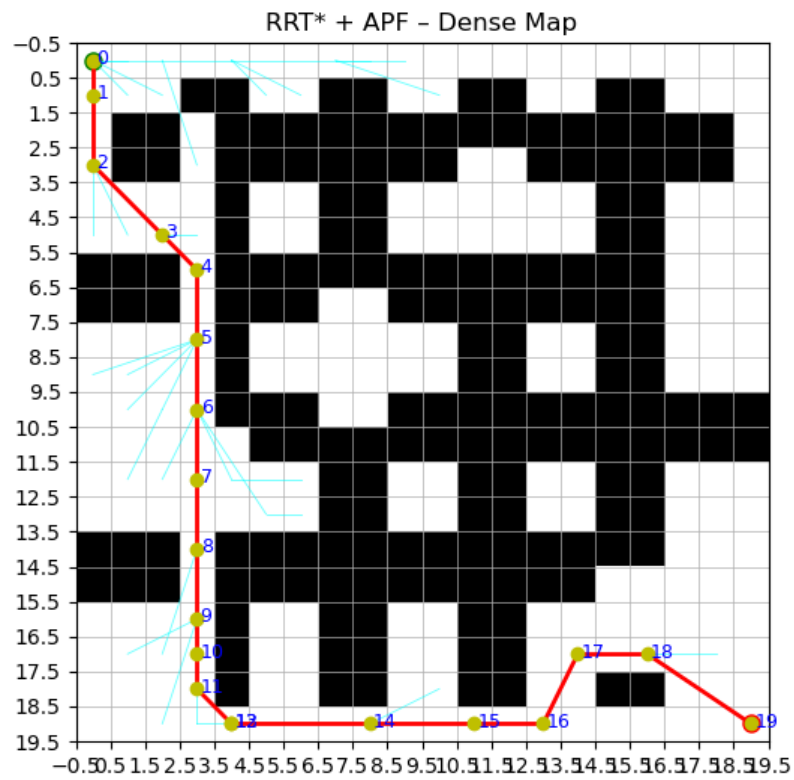
- Optimality of RRT* is evident paths are shorter than RRT for same grids.

4.2f RRT*+ APF Algorithm

1) 20x20 grid

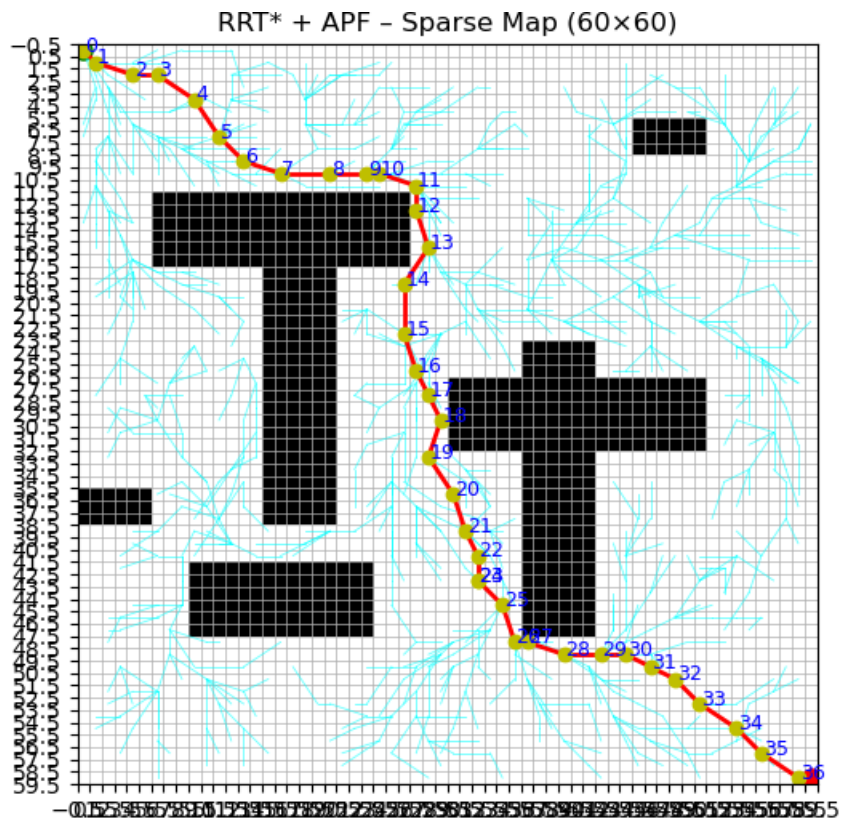
1.(a) Sparse Obstacles Mid Obstacles and Dense Obstacles



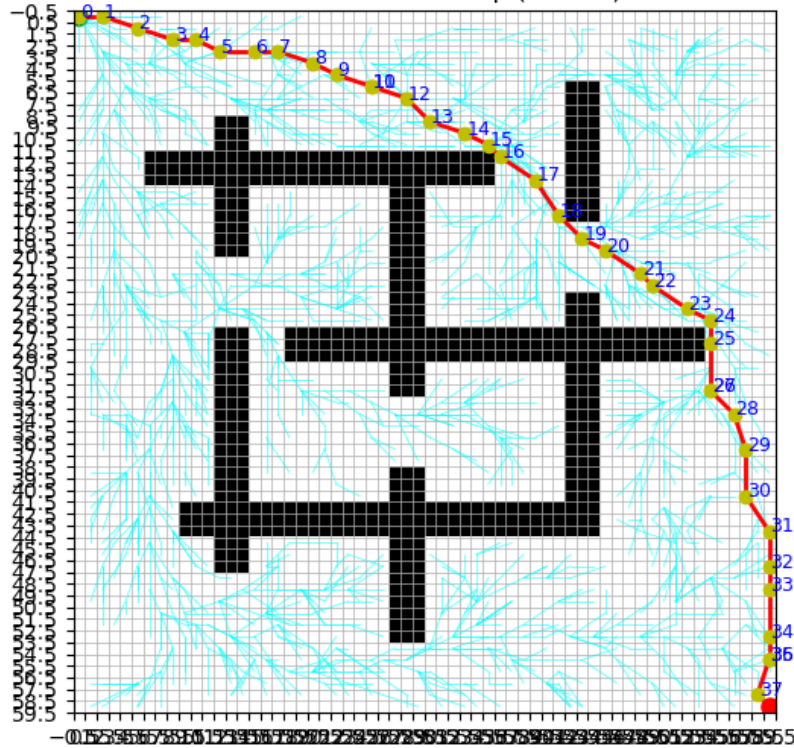


2) 60x60 grid

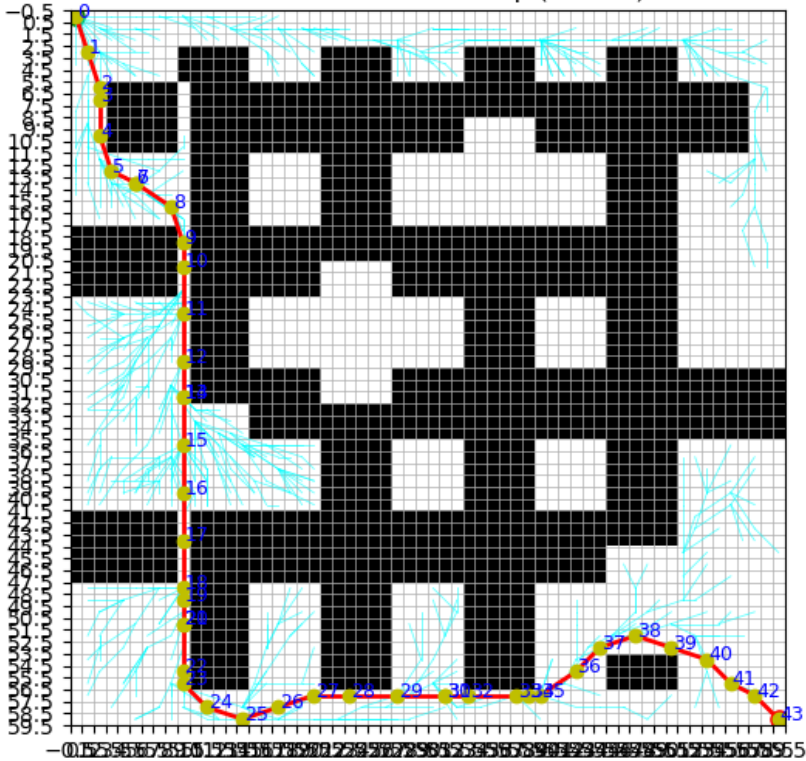
2.(a) Sparse Obstacles Mid Obstacles and Dense Obstacles



RRT* + APF - Mid Map (60×60)

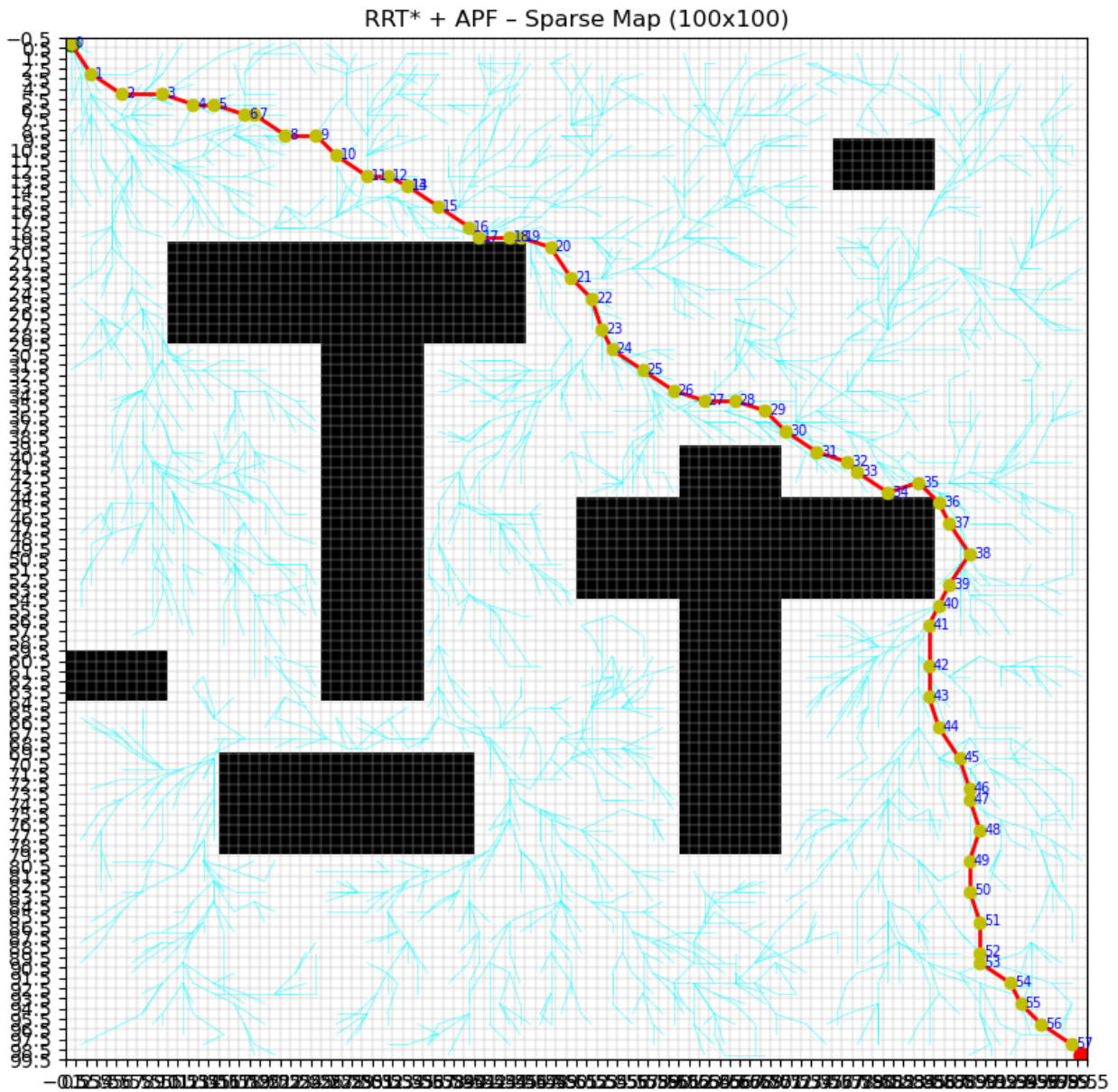


RRT* + APF - Dense Map (60×60)

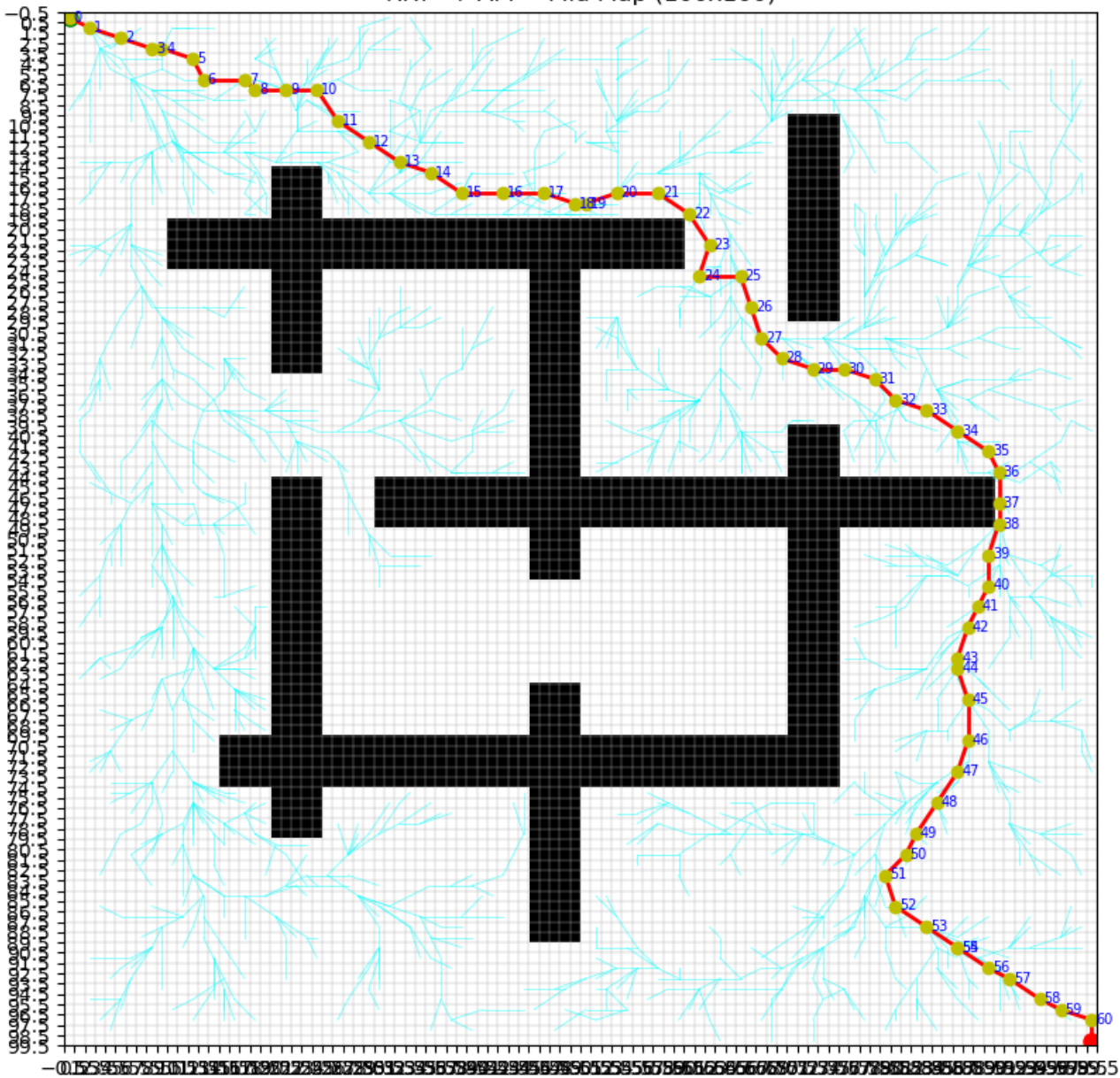


3) 100x100 grid

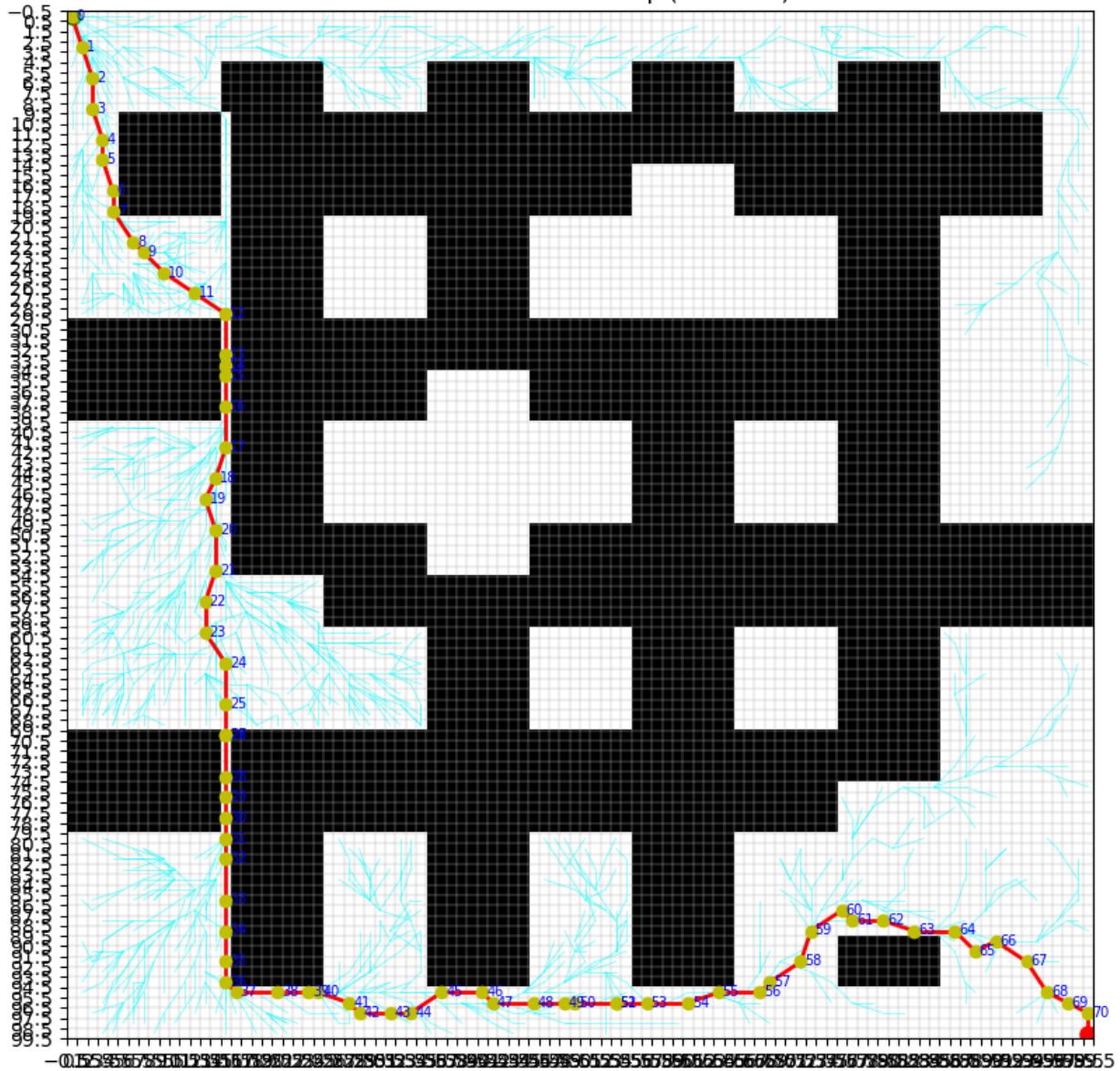
3.(a) Sparse Obstacles Mid Obstacles and Dense Obstacles



RRT* + APF - Mid Map (100x100)



RRT* + APF - Dense Map (100x100)



- Quantitative summary Analysis

1.For sparse obstacle density

Run	20×20 Nodes	20×20 WP	60×60 Nodes	60×60 WP	100×100 Nodes	100×100 WP
1	154	11	1664	37	2034	66
2	104	13	497	36	2493	59
3	72	12	943	36	3306	58
4	45	10	1468	34	4424	65
5	271	11	610	38	2971	65
6	154	11	639	36	2246	60
7	135	12	480	38	1759	65
8	133	12	860	39	941	68
9	101	12	954	37	1953	63
10	62	10	844	40	1910	62
11	106	12	760	37	1773	66
12	127	12	1314	37	2553	62
13	49	10	706	38	1717	61
14	84	12	1362	38	1132	65
15	138	12	704	41	2880	64
16	143	12	1317	35	2152	62
17	106	12	833	39	4431	57
18	205	12	1631	38	2342	60
19	93	12	638	35	1949	63
20	83	11	607	39	1989	64

Number of Nodes (Standard Deviation)

- 20×20: 53.46
- 60×60: 378.37
- 100×100: 906.64

Grid	Best Nodes	Worst Nodes	Best WP	Worst WP
20×20	45	271	10	13
60×60	480	1664	34	41
100×100	941	4431	57	68

2.For mid obstacle density

Run	20×20 Nodes	20×20 WP	60×60 Nodes	60×60 WP	100×100 Nodes	100×100 WP
1	124	14	812	42	2825	61
2	147	11	554	39	1749	62
3	135	11	1072	40	2558	70
4	92	13	987	38	2374	65
5	41	12	1157	39	1547	64
6	59	11	715	40	1644	72
7	175	12	578	39	1235	66
8	307	16	1057	41	1713	68
9	159	12	553	42	2490	63
10	274	12	1575	36	1456	66
11	79	12	1001	39	2523	61
12	249	13	1067	40	1537	61
13	90	12	809	42	1803	63
14	81	12	676	37	1609	67
15	83	12	1442	43	2182	65

Run	20×20 Nodes	20×20 WP	60×60 Nodes	60×60 WP	100×100 Nodes	100×100 WP
16	195	12	1295	41	4500	59
17	157	11	970	39	1736	69
18	180	11	617	37	3336	67
19	129	13	1087	37	3582	63
20	105	11	1704	40	1937	62

Number of Nodes (Standard Deviation)

- 20×20: 71.57
- 60×60: 335.03
- 100×100: 830.80

Grid	Best Nodes	Worst Nodes	Best WP	Worst WP
20×20	41	307	11	16
60×60	553	1704	36	43
100×100	1235	4500	59	72

3.For dense obstacle density

Run	20×20 Nodes	20×20 WP	60×60 Nodes	60×60 WP	100×100 Nodes	100×100 WP
1	87	19	588	44	2020	76
2	169	18	707	42	1733	72
3	95	16	1822	47	2084	76
4	127	17	1242	46	3183	73
5	118	14	998	47	4333	84
6	199	17	1394	47	1536	76
7	191	17	1533	46	7563	81

Run	20×20 Nodes	20×20 WP	60×60 Nodes	60×60 WP	100×100 Nodes	100×100 WP
8	144	16	1800	48	834	81
9	533	15	1191	50	2424	73
10	151	19	1017	44	1555	77
11	130	16	428	46	2047	79
12	200	17	1363	46	2586	78
13	92	17	613	43	1218	79
14	152	16	553	48	5703	76
15	82	16	1162	46	4635	83
16	171	17	1170	48	4150	75
17	138	15	1958	51	1519	78
18	216	19	1672	47	2393	79
19	104	17	941	45	2170	72
20	90	17	562	47	3661	76

Number of Nodes (Standard Deviation)

- 20×20: 97.19
- 60×60: 468.26
- 100×100: 1688.72

Grid	Best Nodes	Worst Nodes	Best WP	Worst WP
20×20	82	533	14	19
60×60	428	1958	42	51
100×100	834	7563	72	84

1.RRT*+APF Analysis

1.1RRT*+ APF – Sparse Maps

Observations (Sparse):

- Node count increases sharply with grid size, as expected due to larger search space.
- Waypoints remain low and consistent, reflecting smooth paths due to APF guidance.
- Variability in nodes is high in larger grids (100×100), showing stochastic tree growth.

1.2RRT* + APF – Mid Maps

Observations (Mid):

- Node counts slightly higher than Sparse due to additional obstacles, but not extreme.
- Waypoints increase slightly → paths become more winding, but still much smoother than RRT.
- Std deviation grows with grid size but remains manageable.

1.3RRT* + APF – Dense Maps

Observations (Dense):

- Node count increases dramatically with grid size and density → dense maps require extensive exploration.
- Waypoints increase, but paths remain smoother than standard RRT due to APF guidance.
- Std deviation of nodes is high in large dense maps, showing that tree growth can vary a lot depending on initial random samples.

5. Discussion

5.1 Performance Comparison

The comparative evaluation of A*, RRT, and APF demonstrates distinct performance characteristics across sparse and dense environments.

A* consistently produced shortest-path solutions due to its heuristic-driven graph search mechanism, yielding optimal path lengths in sparse and moderately sized grids. However, its **time complexity** escalates as $O(b^d)$, where (b) is the branching factor and (d) is the depth of the solution, making it computationally expensive in dense grids or large-scale environments (e.g., 100×100).

RRT exhibited **high scalability** in continuous or high-dimensional spaces, maintaining goal reachability even when the obstacle configuration is dense; the primary compromise was irregular path and the need for high iteration counts (e.g., 10,000) to ensure full coverage.

APF provided rapid convergence in sparse maps due to its continuous potential field formulation, with computation largely independent of grid size, but was susceptible to **local minima traps**, oscillatory behaviour near narrow passages, and failed to reach goals in highly dense regions.

RRT* improves upon RRT by introducing optimal parent selection and rewiring, resulting in asymptotically optimal paths. It produces shorter and smoother trajectories compared to RRT and reliably reaches the goal in both sparse and moderately dense grids. However, its computational cost is higher due to the neighbourhood search and rewiring operations, especially in large-scale grids.

RRT* + APF integrates artificial potential field guidance into RRT*, accelerating convergence and generating smoother paths. This hybrid approach reduces path length and increases efficiency in sparse and moderately dense environments. Despite these improvements, APF-induced local minima can still affect performance in highly cluttered or narrow obstacle regions.

5.2 Algorithm Behaviour

1. **A***: Behaviour is governed by the evaluation function $f(n) = g(n) + h(n)$, combining actual cost $(g(n))$ and heuristic $(h(n))$. The algorithm deterministically expands nodes with minimal $(f(n))$, producing consistent paths but being because it is highly sensitive to grid resolution and heuristic admissibility. High-density obstacle regions increase the frequency of backtracking, leading to longer computation times.
2. **RRT**: Exhibits exploration by incrementally growing a tree from the start toward random samples. Its non-deterministic nature enables coverage in complex, high-dimensional spaces but produces irregular paths, which often require post processing to reduce path length and improve feasibility. RRT behaviour is strongly influenced by step size and maximum iterations, affecting convergence rate and success probability.
3. **APF**: Uses continuous attractive forces toward the goal and repulsive forces from obstacles, generating a smooth and reactive trajectory. Algorithmic behaviour depends on

the relative magnitudes of (K_{att}) and (K_{rep}), repulsive radius, and step size. Oscillations occur when attractive and repulsive forces are comparable, especially in narrow corridors, and local minima can prevent goal attainment without additional strategies such as random perturbations or potential field modification.

4. **RRT***: Behavior is similar to RRT but includes optimal parent selection and rewiring. Nodes are added with the lowest-cost parent, and nearby nodes are rewired to reduce overall path cost. This ensures smoother, shorter, and more consistent paths. The algorithm converges asymptotically toward the optimal solution but requires more computation due to neighborhood searches.
5. **RRT* + APF**: Combines RRT*'s rewiring mechanism with APF guidance. Nodes are biased toward the goal and away from obstacles, accelerating convergence and producing smoother paths than RRT* alone. Behavior is influenced by APF parameters (K_{att} , K_{rep} , obstacle radius) in addition to step size and rewiring radius. While effective in sparse and moderately dense maps, local minima induced by repulsive forces can still slow progress or trap the tree in highly cluttered areas.

5.3 Limitations and Trade-offs

1. **A***: Limited by computational and memory overhead in large grids; cannot inherently handle continuous spaces or real-time replanning without modifications.
2. **RRT**: Randomized exploration ensures probabilistic completeness but lacks path optimality; increased iterations improve coverage at the expense of computational time. Sensitive to step size, where large steps can miss narrow passages, and small steps increase runtime.
3. **APF**: Highly sensitive to environmental clutter; local minima, oscillations, and scenarios where goal are not reached are common. Parameter tuning (force coefficients, step size) is critical for stability, but no global optimality guarantees exist.
4. **RRT***: Improves path optimality over RRT through rewiring and cost-based parent selection, producing smoother and shorter paths. However, it incurs higher computational cost due to neighborhood searches and rewiring operations, especially in large or dense environments.
5. **RRT* + APF**: Combines RRT*'s optimal path generation with APF guidance for faster convergence and smoother paths. Still susceptible to APF local minima in highly cluttered regions, requiring careful tuning of attractive and repulsive coefficients. Computational cost remains higher than RRT due to combined rewiring and potential field calculations.
6. **Trade-offs**:
 - a) **A*** maximizes optimality in discrete grids but struggles in large or dense maps.
 - b) **RRT** balances exploration and feasibility in complex environments but yields suboptimal paths.
 - c) **APF** emphasizes reactive and computationally light navigation but can fail in dense or narrow spaces.

- d) **RRT*** prioritizes path optimality and smoothness at the expense of higher computation.
- e) **RRT* + APF** accelerates convergence and improves path quality but still faces local minima and increased computational overhead.

5.4 Observations

1. Sparse environments:

APF outperformed in computation speed due to continuous, reactive updates. A* achieved the most optimal paths. RRT reliably reached the goal but generated non-smooth paths. RRT* produced smoother and shorter paths than RRT, with moderate computation time. RRT* + APF further reduced path length and improved convergence speed while maintaining smooth trajectories.

2. Dense environments:

APF frequently failed due to local minima. RRT's probabilistic sampling allowed eventual convergence with sufficient iterations, but paths were irregular. A* faced backtracking overhead and occasional failure in 100×100 grids. RRT* reliably reached the goal and generated more optimal paths than RRT, but required higher computation due to rewiring. RRT* + APF improved convergence and path smoothness, though APF-induced local minima could still occur in highly cluttered regions.

3. Parameter influence:

Step size and maximum iterations were critical for RRT and RRT*. Attractive/repulsive coefficients and repulsion radius dominated APF and RRT* + APF behaviour. Heuristic design and grid granularity influenced A* efficiency. Rewiring radius in RRT* and RRT* + APF affected path optimality and computational load.

4. Practical insight:

A* is suited for medium-sized, discrete grids where optimality is paramount. RRT is appropriate for high-dimensional or continuous environments requiring exploration. APF is ideal for real-time, reactive path planning in open or sparsely obstructed spaces. RRT* balances optimality and feasibility, suitable for complex maps needing smoother paths. RRT* + APF excels in sparse to moderately dense environments, combining rapid convergence, smooth paths, and global exploration, but still requires careful parameter tuning in highly cluttered areas.

6.CONCLUSION

This work studies and compares, three widely used path planning algorithms in UAVs —A*, Rapidly-exploring Random Tree (RRT), and Artificial Potential Field (APF)—were implemented and evaluated in a 2-Dimensional environment containing multiple fixed obstacles. All algorithms were tested on the same configuration space (map and obstacle layout) generated from an obstacle map so that their performances could be directly compared under identical conditions.

The A* algorithm showed predictable and optimal behavior like consistently finding the shortest path between the start position and goal position, ensuring the path is collision free. From the plotted results, it is seen that A* explores a large portion of the search space, particularly near the obstacles, which guarantees optimality but increases the computational cost. Hence, A* is mostly suited for offline planning or well-structured environments where finding the best possible path is more important than the computation speed.

The RRT algorithm successfully explored the environment picking random nodes/points and started building tree from the start. Furthermore, it quickly found a feasible path even in the presence of complex obstacle layout. The tree-based exploration enabled efficient coverage of the free space with less computations relatively. However, the resulting paths were not optimal and irregular, which will then require post-processing for smoothing. RRT is therefore best suitable for situations where the environment is not known and complicated and rapid feasibility is more important than optimality.

The APF method generated a smooth and continuous trajectory path by combining attractive and repulsive forces (resultant of the forces). The method is fast and works very well in case of real time navigation. However the UAVs can get stuck in a situation where both the forces cancel out each other and creates a local minimum which prevents the system from reaching the goal. Hence proper tuning methods are required in case of applying APF to the environment.

The RRT* algorithm improves upon RRT by incorporating cost-based rewiring and optimal parent selection, producing smoother and shorter paths that are closer to optimal. Although computationally heavier than RRT, RRT* provides a good balance between path quality and exploration, making it suitable for moderately complex environments requiring both feasibility and path efficiency.

The RRT* + APF hybrid further enhances RRT* by integrating potential field guidance into the sampling process. This approach accelerates convergence, generates smoother trajectories, and reduces path length, particularly in sparse and moderately dense environments. However, the method can still encounter APF-induced local minima in highly cluttered areas and requires careful parameter tuning.

Concluding, the results indicate that no single algorithm is universally optimal for all UAV path planning scenarios. A* offers optimal paths at higher computational cost, RRT provides fast feasible solutions in complex environments, and APF enables smooth real-time motion but requires careful tuning, RRT* balances optimality and feasibility, and RRT* + APF combines smooth, fast, and near-optimal path planning with global exploration capabilities. Algorithm selection should therefore depend on mission requirements, including environment complexity, computational resources, and real-time execution needs. Future work may focus on hybrid approaches, such as combining global and reactive planners to overcome local minima while maintaining path optimality.

The choice of algorithm should therefore depend on mission requirements, including environment complexity, computational constraints, and real-time execution needs. A* is suited for structured environments where optimality is critical, RRT for unknown or highly complex spaces requiring rapid feasible paths, APF for reactive, real-time navigation in open or sparsely obstructed areas, RRT* for balancing path optimality and feasibility, and RRT* + APF for fast, smooth, and near-optimal trajectories with global exploration capabilities. Future work may focus on hybrid approaches, such as combining RRT* with A* or APF for improved path quality and smoothness, or enhancing RRT* + APF with global planners to overcome local minima while maintaining fast and reliable trajectory generation.

7.FUTURE WORK

1. Algorithm Enhancement

- a) A*: Implement dynamic or weighted A* to reduce computation in large or dense grids and improve real-time applicability. Incorporate adaptive heuristics to better handle clustered obstacles.
- b) RRT: Explore RRT* or *RRT-Smart** variants to generate smoother, near-optimal paths and reduce post-processing requirements. Investigate bidirectional RRT for faster convergence in complex maps.
- c) APF / RRT + APF*: Integrate local minima escape strategies, such as randomized perturbations or hybrid potential fields, to improve success rates in dense obstacle environments. Experiment with adaptive step sizes and force coefficients for enhanced stability.

2. Hybrid Approaches

- a) Combine APF or RRT* + APF with RRT or A* to leverage APF's reactive navigation for short-term obstacle avoidance while using RRT / RRT* / A* for global path planning.
- b) Implement multi-layered planning with A* for coarse global paths and APF or RRT* + APF for fine local trajectory adjustments.

3. Realistic Environment Testing

- a) Extend experiments to continuous or high-dimensional spaces, including non-uniform obstacle shapes and dynamic obstacles.
- b) Test algorithms on robotic simulators or real robots to validate performance under real-world constraints, including sensor noise and kinematic limitations.

4. Performance Optimization

- a) Analyse parallelization strategies for A* and RRT to reduce computation time.
- b) Implement adaptive sampling in RRT and RRT* to improve efficiency in dense regions.
- c) Study parameter sensitivity for APF and RRT*+APF to develop automated tuning mechanisms for various environment types.

5. Integration with Learning-Based Methods

- a) Explore reinforcement learning or neural network-guided heuristics to dynamically improve path selection for A*, RRT, and APF.

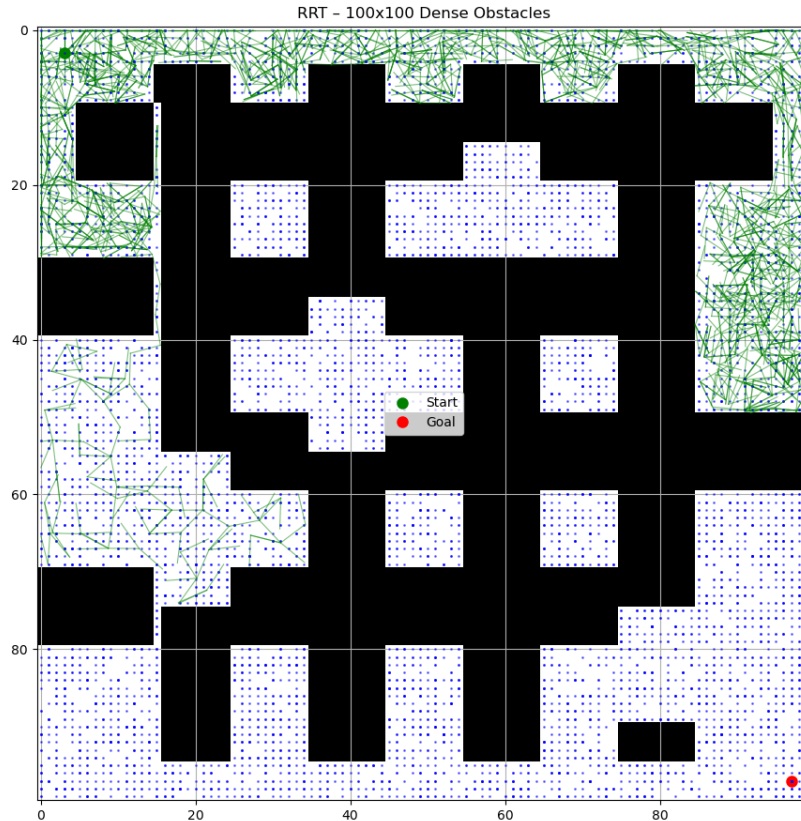
- b) Investigate predictive obstacle avoidance using AI to enhance APF and RRT performance in dynamic environments.

8. REFERENCES

- 1) F. Desiderio, "Development of a modified A-Star Algorithm for path planning," M.S. thesis, Automation and Control Engineering.
- 2) Materials and papers consulted via ResearchGate.
- 3) Video lectures and tutorials consulted via YouTube.

9. APPENDICES

9.1 RRT: 100x100 (Dense)



In this figure, the goal position was not reached

Possibilities:

The RRT may not have sampled nodes that could connect to the goal within 10,000 iterations.

In the narrow passages, the step may have been too large, causing the tree to “skip over” feasible connections to the goal.